

DESIGN, INTEGRATION, AND VERIFICATION OF A RISC-V-BASED SYSTEM-ON-CHIP

Scope-Controlled Hardware Specification for Kiro

VeeR EL2 RV32IMC Processor + 64 KB IMEM + 64 KB DMEM + UART + Timer + GPIO + SPI + I2C +
PWM

Document item	Specification
Primary purpose	Define exactly what Kiro shall implement and prevent unsupported functionality from entering the design.
Processor	VeeR EL2, RV32IMC, machine-mode-only, four-stage scalar in-order pipeline.
Processor-side baseline bus	AHB-Lite configuration for the baseline implementation.
Peripheral-side bus	APB-style interface: PCLK, PRESETn, PADDR, PSEL, PENABLE, PWRITE, PWDATA, PRDATA, PREADY.
Project memories	64 KB IMEM at 0x0000_0000-0x0000_FFFF; 64 KB DMEM at 0x0001_0000-0x0001_FFFF.
Project peripherals	UART, Timer, GPIO, SPI, I2C, PWM.
Project interrupts	TMR_IRQ, UART_RX_IRQ, SPI_DONE_IRQ, I2C_DONE_IRQ only.
Baseline out-of-scope items	Extra peripherals, extra CPU cores, second buses, project DMA controller, external PLIC, GPIO/PWM interrupts, undocumented protocol features.

Prepared from the supplied project specification and VeeR EL2 reference material.

DOCUMENT CONTROL AND SOURCE-OF-TRUTH RULE

Purpose of this revision

The supplied working document had grown to approximately 1,524 rendered pages because the same block concepts were repeated in many separate subsections. This revision consolidates the architecture into a controlled implementation specification: one source-of-truth address map, one signal map, one register map, one interrupt model, one control/data path definition, and one explicit scope boundary. The goal is not to reduce technical depth; it is to remove repetition and eliminate ambiguity.

Authoritative project decisions

- Use VeeR EL2 as the processor. Do not replace it with a generic RISC-V CPU.
- Use the selected VeeR AHB-Lite processor-side configuration for the baseline implementation; do not implement AXI4 and AHB-Lite simultaneously unless the project explicitly changes.
- Keep IFU and LSU system-bus interfaces logically separate.
- Use the APB-style peripheral interface on the project peripheral side and connect it through a bus-adaptation layer.
- Keep the fixed 32-bit project address map unchanged.
- Implement only UART, Timer, GPIO, SPI, I2C and PWM as project peripherals.
- Use only the four defined peripheral interrupt sources in the baseline interrupt integration.
- Treat project peripheral addresses as side-effecting I/O rather than ordinary cacheable memory.
- Every VeeR system-bus request must receive a defined response; unmapped requests must not hang the processor.
- Do not add functionality merely because VeeR EL2 is capable of supporting it.

SCOPE NOTE: When a later section conflicts with an earlier address, register, signal, interrupt, reset, or scope statement, the fixed project memory map, signal table, register tables, interrupt list, and scope rules in this document take precedence.

1. DETAILED INTRODUCTION

1.1 System Overview

This project is a RISC-V-based embedded System-on-Chip built around the VeeR EL2 processor. The SoC combines a processor core, separate instruction and data memories, a system interconnect, a processor-to-peripheral bus adaptation layer, and six project peripherals: UART, Timer, GPIO, SPI, I2C and PWM. The complete design is intended to expose the hardware path from instruction fetch through register access to external-device control, while keeping every block and interface explicitly defined.

1.2 VeeR EL2 Processor Selection

VeeR EL2 is the fixed processor for this project. The supplied VeeR reference material describes it as a 32-bit RV32IMC, machine-mode-only processor with a four-stage scalar in-order pipeline and dedicated IFU, DEC, EXU and LSU functional units. The M extension provides multiply/divide support, the C extension provides compressed instructions, and the processor includes CSR and other documented VeeR control facilities. The project must use the actual VeeR interfaces rather than assuming a generic RISC-V bus.

1.3 Processor Internal Organization

The four major functional units are the Instruction Fetch Unit (IFU), Decode Unit (DEC), Execution Unit (EXU) and Load/Store Unit (LSU). The IFU obtains instructions through the instruction-side master interface. The DEC interprets instruction encodings and produces control decisions. The EXU performs arithmetic, logical, branch and related execution functions. The LSU performs load/store accesses and is therefore the main path used by software to reach DMEM and memory-mapped peripherals.

1.4 Processor-Side versus Peripheral-Side Bus

VeeR EL2 does not expose a native generic APB master for software accesses. Its system interfaces are configurable as AXI4 or AHB-Lite. The project peripheral architecture, however, is defined using an APB-style interface with PADDR, PSEL, PENABLE, PWRITE, PWDATA, PRDATA and PREADY. Therefore the SoC must maintain two explicit bus levels: VeeR processor-side AHB-Lite and project peripheral-side APB-style. A required bus-adaptation layer translates between them without adding application functionality.

1.5 Memory Architecture

The project uses separate 64 KB Instruction Memory and 64 KB Data Memory. IMEM occupies 0x0000_0000-0x0000_FFFF and is read during normal execution. DMEM occupies 0x0001_0000-0x0001_FFFF and is read/write. VeeR EL2 also supports optional ICCM, DCCM and I-cache features, but these do not become project blocks automatically. They are outside baseline scope unless explicitly selected in the actual VeeR configuration.

1.6 Memory-Mapped I/O

All project peripherals are accessed using normal RISC-V load/store instructions. A software store reaches the LSU, the system interconnect decodes the address, the correct peripheral select is generated, the bus adapter creates the APB-style transaction, and the selected peripheral updates its register. A read follows the reverse data path. This architecture keeps application software simple while preserving hardware visibility and verification at each layer.

1.7 Project Peripheral Set

The project peripherals are UART, Timer, GPIO, SPI, I2C and PWM. UART provides asynchronous serial communication. Timer provides programmable timing and generates TMR_IRQ. GPIO provides eight general-purpose digital I/O pins. SPI provides synchronous full-duplex master communication. I2C provides standard-mode two-wire master communication with 7-bit addressing and byte-level read/write with ACK/NACK handling. PWM is the custom in-house waveform generator with programmable period and duty comparison.

1.8 Interrupt Architecture

The project defines exactly four peripheral interrupt sources: TMR_IRQ, UART_RX_IRQ, SPI_DONE_IRQ and I2C_DONE_IRQ. These are level-sensitive at the project peripheral level and are connected to the VeeR external-interrupt/PIC mechanism. GPIO and PWM do not generate project interrupts in the baseline design. The four peripheral sources do not receive separate standard mcause values; they are external interrupt sources, and the VeeR PIC/source mechanism identifies the actual source.

1.9 Machine-Level Interrupts

VeeR uses the standard machine interrupt CSRs mie (0x304), mip (0x344) and mcause (0x342). The external interrupt enable is mie[11]=meie, the external interrupt pending bit is mip[11]=meip, and the machine-external-interrupt cause is 0x8000_000B. These are processor-level mechanisms and shall not be reimplemented in UART, Timer, GPIO, SPI, I2C or PWM.

1.10 NMI and Processor-Level Errors

VeeR also provides a separate NMI mechanism through nmi_int and nmi_vec. Documented processor-level NMI causes include machine D-bus store error, machine D-bus non-blocking load error, fast-interrupt double-bit ECC error, DCCM region access error and non-DCCM NMI conditions. Ordinary peripheral conditions such as an I2C NACK, SPI BUSY, UART RX condition or Timer compare event are not VeeR NMI causes and must remain at the peripheral status/interrupt level.

1.11 Side-Effecting I/O

The project peripheral range 0x0002_0000-0x0002_05FF consists of hardware registers whose reads and writes can produce side effects. The VeeR architecture identifies system-bus I/O as non-cacheable and side-effecting. The integration therefore must treat the peripheral region as I/O and preserve the required ordering/access semantics.

1.12 Speculative Instruction Access

VeeR EL2 can generate speculative instruction-side accesses because of instruction-cache misses, exceptions and prediction mechanisms. The supplied VeeR material also states that speculative data accesses are not issued on the LSU interface. The IFU/interconnect path must therefore tolerate valid instruction-side requests while the LSU remains the controlled load/store path for DMEM and memory-mapped peripherals.

1.13 Mandatory Response Rule

A central system constraint is that every VeeR system-bus request must receive a defined response. The VeeR documentation warns that an unhandled request can leave the processor hung. The interconnect must therefore respond to valid memory requests, valid peripheral requests, unmapped requests and error conditions rather than leaving any unsupported address permanently outstanding.

1.14 Unmapped Address Handling

The project reserves 0x0002_0600-0xFFFF_FFFF. For an unmapped project address, the supplied architecture defines PRDATA=0x0000_0000 and PREADY asserted after one wait cycle on the APB-style side so that the bus cannot lock. The processor-side AHB-Lite response must be converted into the corresponding VeeR-compatible completion/error semantics.

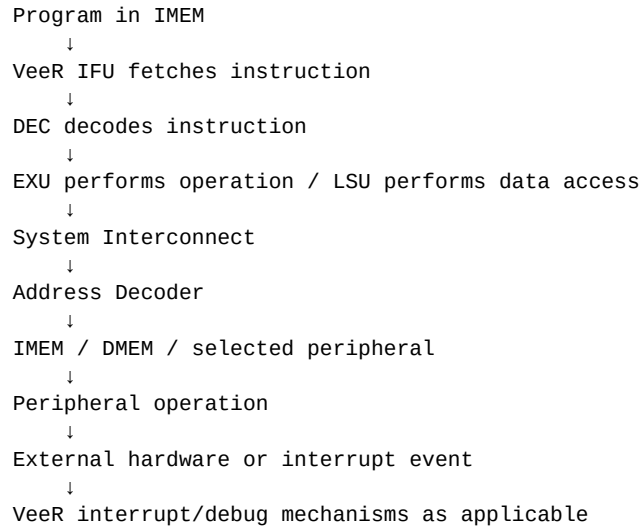
1.15 Reset and Boot

The core complex uses clk, rst_l and rst_vec[31:1], with dbg_rst_l for the Debug Module. Project peripherals use PCLK and PRESETn. The core clock must be stable before rst_l is deasserted. The reset vector must be configured consistently with the project boot memory at 0x0000_0000. Active SPI/I2C transactions are aborted by peripheral reset, and reset must return peripheral control/status state to defined defaults.

1.16 Debug, DMA and Optional VeeR Features

VeeR EL2 includes Debug and a Debug Bus Master and also defines a DMA slave interface and optional ICCM/DCCM/I-cache. These are processor capabilities, not automatic project subsystems. The baseline project does not add a project DMA controller, additional CPU cores, an external PLIC, or extra peripherals. VeeR Debug remains processor-support infrastructure.

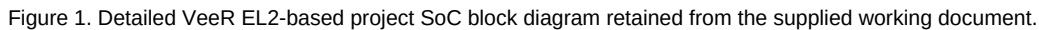
1.17 Overall System Flow

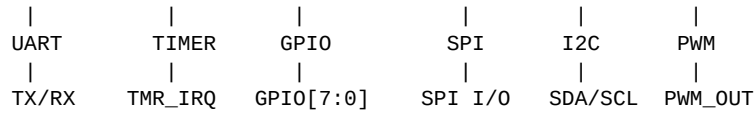


1.18 Introduction Scope Lock

SCOPE NOTE: This introduction is the architectural baseline for all later sections. Later register, signal, control-path, data-path, error and verification definitions must map back to the blocks and addresses defined here. Unsupported VeeR features are not project requirements merely because they exist in the processor.

2.1 Normalized Block Diagram





Interrupt sources: TMR_IRQ, UART_RX_IRQ, SPI_DONE_IRQ, I2C_DONE_IRQ
→ VeeR External Interrupt / PIC → VeeR EL2

2.2 Block Inventory

#	Block	Role
1	VeeR EL2 Processor Core	Central RV32IMC machine-mode-only processor.
2	Instruction Fetch Unit (IFU)	Instruction-side request generation and instruction return.
3	Decode Unit (DEC)	Instruction interpretation and control generation.
4	Execution Unit (EXU)	Arithmetic, logic, branch and execute-stage operations.
5	Load/Store Unit (LSU)	Data-memory and memory-mapped I/O load/store generation.
6	IFU AHB-Lite Master Interface	Processor instruction-side AHB-Lite signals.
7	LSU AHB-Lite Master Interface	Processor data-side AHB-Lite signals.
8	System Interconnect	Routes VeeR system transactions to resources.
9	Address Decoder	Maps the fixed system address space to targets.
10	Response MUX	Returns selected read/ready/response information.
11	AHB-Lite to APB-Style Bus Adapter	Translates processor-side transaction semantics to peripheral-side APB-style signals.
12	APB-Style Peripheral Bus	Common PCLK/PRESETn/PADDR/PSEL/PENABLE/PWRITE/PWDATA/PRDATA/PREADY interface.
13	Instruction Memory (IMEM)	64 KB boot-loaded instruction storage.
14	Data Memory (DMEM)	64 KB read/write data storage.
15	UART Controller	Asynchronous serial communication.
16	Timer Controller	Programmable count/compare timing source and TMR_IRQ.
17	GPIO Controller	Eight-bit general-purpose digital I/O.
18	SPI Controller	Full-duplex synchronous serial master.
19	I2C Controller	Standard-mode two-wire master, 7-bit addressing, byte read/write.
20	PWM Generator	Custom programmable-period/duty waveform generator.
21	VeeR PIC	Core-integrated external interrupt management.
22	VeeR Debug/JTAG	Processor-support debug transport and hart control.
23	Clock and Reset Connectivity	clk/PCLK/rst_l/PRESETn/dbg_rst_l and related enables/reset-vector integration.

2.3 Address Map

Target	Start	End	Size	Select
IMEM	0x0000_0000	0x0000_FFFF	64 KB	imem_sel
DMEM	0x0001_0000	0x0001_FFFF	64 KB	dmem_sel
UART	0x0002_0000	0x0002_00FF	256 B	psel[0]
Timer	0x0002_0100	0x0002_01FF	256 B	psel[1]
GPIO	0x0002_0200	0x0002_02FF	256 B	psel[2]
SPI	0x0002_0300	0x0002_03FF	256 B	psel[3]
I2C	0x0002_0400	0x0002_04FF	256 B	psel[4]
PWM	0x0002_0500	0x0002_05FF	256 B	psel[5]
Reserved	0x0002_0600	0xFFFF_FFFF	Reserved	none

2.4 Architectural Boundaries

- VeeR EL2 is the processor boundary. Peripheral logic must not be embedded into IFU/DEC/EXU/LSU.
- System Interconnect and Address Decoder route transactions; they do not implement peripheral protocols.
- Bus Adapter performs protocol/width/control translation; it does not add application functions.
- APB-style bus is the common project peripheral interface.
- PIC handles external interrupt management; peripheral registers remain the source of detailed peripheral status.
- Debug is processor-support infrastructure and is separate from normal LSU peripheral accesses.

3. COMPLETE SIGNAL LIST

3.1 Signal Naming Rules

Signal names below are intentionally frozen for the specification. Where the selected VeeR RTL uses exact processor-port names, those VeeR names are authoritative. Internal equivalent names may be used only when the observable behaviour and connectivity remain identical.

3.2 Top-Level Clock, Reset and Core Identification

Signal	Width	Direction at integration boundary	Meaning
clk	1	Input	VeeR core-complex clock.
rst_l	1	Input	Active-low VeeR core-complex reset excluding Debug Module.
rst_vec[31:1]	31	Input	VeeR reset vector.
dbg_rst_l	1	Input	Debug Module/JTAG synchronizer reset.
PCLK	1	Input	Project peripheral-bus clock.
PRESETn	1	Input	Active-low project peripheral reset.
core_id[31:4]	28	Input/config	Upper portion of VeeR mhartid in the single-hart configuration.

3.3 VeeR IFU AHB-Lite Signals

Signal	Width	Role
haddr	32	Instruction address
hburst	3	Burst information
hmastlock	1	Master lock
hprot	4	Protection/control attributes
hsize	3	Transfer size
htrans	2	Transfer type
hwrite	1	Read/write direction
hrdata	64	Instruction read data
hready	1	Instruction transaction completion
hresp	1	Instruction response

3.4 VeeR LSU AHB-Lite Signals

Signal	Width	Role
lsu_haddr	32	Data address
lsu_hburst	3	Burst information
lsu_hmastlock	1	Master lock
lsu_hprot	4	Protection/control attributes
lsu_hsize	3	Transfer size
lsu_htrans	2	Transfer type
lsu_hwdata	64	Write data
lsu_hwrite	1	Read/write direction
lsu_hrdata	64	Returned read data
lsu_hready	1	Data transaction completion
lsu_hresp	1	Data response

3.5 VeeR Bus Clock Enables

Signal	Width	Meaning
ifu_bus_clk_en	1	Instruction bus clock-enable control
lsu_bus_clk_en	1	LSU bus clock-enable control
dbg_bus_clk_en	1	Debug bus clock-enable control when debug bus is instantiated
dma_bus_clk_en	1	VeeR DMA-interface clock-enable control; not an active project DMA subsystem

3.6 Project Peripheral Bus Signals

Signal	Width	Direction	Meaning
PCLK	1	Input	Peripheral/system clock
PRESETn	1	Input	Active-low synchronous peripheral reset
PADDR[31:0]	32	Input	Peripheral register address
PSEL	1	Input	Selected peripheral
PENABLE	1	Input	ACCESS phase indication
PWRITE	1	Input	1=write, 0=read
PWDATA[31:0]	32	Input	Peripheral write data
PRDATA[31:0]	32	Output	Peripheral read data
PREADY	1	Output	Peripheral transaction completion

3.7 Peripheral Selects

Select	Peripheral	Address window
psel[0]	UART	0x0002_0000-0x0002_00FF
psel[1]	Timer	0x0002_0100-0x0002_01FF
psel[2]	GPIO	0x0002_0200-0x0002_02FF
psel[3]	SPI	0x0002_0300-0x0002_03FF
psel[4]	I2C	0x0002_0400-0x0002_04FF
psel[5]	PWM	0x0002_0500-0x0002_05FF

3.8 Peripheral External Signals

Block	External signals	Width
UART	TX, RX	1 each
Timer	TMR_IRQ	1
GPIO	GPIO[7:0]	8
SPI	SCLK, MOSI, MISO, CS_n	1 each
I2C	SDA, SCL	1 each
PWM	PWM_OUT	1

3.9 Project Interrupt Sources

Signal	Source	Type	Path
TMR_IRQ	Project Timer	Level-sensitive	Timer → VeeR external interrupt/PIC
UART_RX_IRQ	UART receive event	Level-sensitive	UART → VeeR external interrupt/PIC
SPI_DONE_IRQ	SPI transfer complete	Level-sensitive	SPI → VeeR external interrupt/PIC
I2C_DONE_IRQ	I2C transfer complete	Level-sensitive	I2C → VeeR external interrupt/PIC

3.10 Debug AHB-Lite Signals

Signal	Width	Role
sb_haddr	32	Debug system-bus address
sb_hburst	3	Burst information
sb_hmastlock	1	Master lock
sb_hprot[3:0]	4	Protection/control
sb_hsize[2:0]	3	Transfer size
sb_htrans[1:0]	2	Transfer type
sb_hwdata	64	Debug write data
sb_hwrite	1	Read/write
sb_hrdata	64	Debug read data
sb_hready	1	Completion
sb_hresp	1	Response

3.11 VeeR Interrupt/Trap Signals

Signal/CSR	Width	Role
extintsrc_req[...]	Implementation-defined source vector	VeeR external interrupt source interface
timer_int	1	VeeR standard machine-timer interrupt input; not TMR_IRQ
soft_int	1	VeeR software interrupt input
nmi_int	1	VeeR non-maskable interrupt input
nmi_vec[31:1]	31	NMI vector
mie	32	Machine interrupt-enable CSR, CSR 0x304
mip	32	Machine interrupt-pending CSR, CSR 0x344
mcause	32	Machine cause CSR, CSR 0x342

3.12 Signal Width Rules

Category	Width
Processor address	32
Project peripheral address	32
Project peripheral write data	32
Project peripheral read data	32
VeeR AHB read/write data	64
GPIO	8
UART TX/RX	1 each
SPI signals	1 each
I2C SDA/SCL	1 each
PWM_OUT	1
VeeR reset vector	31
VeeR NMI vector	31
core_id[31:4]	28

3.13 Signals That Shall Not Be Confused

- clk vs PCLK: core-complex clock versus project peripheral clock.
- rst_l vs PRESETn: VeeR core-complex reset versus project peripheral reset.
- timer_int vs TMR_IRQ: VeeR standard machine timer versus project Timer IRQ.
- extintsrc_req[...] vs UART_RX_IRQ/SPI_DONE_IRQ/I2C_DONE_IRQ/TMR_IRQ: VeeR interrupt input versus project source signals.
- mip.meip vs SPI_DONE_IRQ (and other project IRQs): processor-level pending bit versus peripheral signal.
- sb_h* debug bus signals vs lsu_h*: Debug Bus Master versus LSU Bus Master.

4. COMPLETE REGISTER MAP AND DESCRIPTIONS

4.1 Register Access Rule

Project peripheral registers are accessed through the LSU using normal RISC-V load/store instructions. The system address selects the peripheral, the local offset selects the register, and the Bus Adapter converts the transaction into the APB-style interface. Exact bit behaviour below follows the supplied working specification.

UART Registers (0x0002_0000-0x0002_00FF)

Offset	Register	Access	Description
0x00	UART_TXDATA	R/W	Transmit byte/data written into UART transmit path.
0x04	UART_RXDATA	R	Received byte/data for software read.
0x08	UART_STATUS	R	bit0 TX_BUSY; bit1 RX_VALID.
0x0C	UART_BAUD	R/W	Baud-rate configuration register.

Timer Registers (0x0002_0100-0x0002_01FF)

Offset	Register	Access	Description
0x00	TMR_COUNT	R/W	Timer count value.
0x04	TMR_COMPARE	R/W	Compare threshold.
0x08	TMR_CTRL	R/W	bit0 ENABLE; bit1 IRQ_EN.

GPIO Registers (0x0002_0200-0x0002_02FF)

Offset	Register	Access	Description
0x00	GPIO_OUT	R/W	Output data register.
0x04	GPIO_IN	R	Input pin state.
0x08	GPIO_DIR	R/W	bit=1 output, bit=0 input.

SPI Registers (0x0002_0300-0x0002_03FF)

Offset	Register	Access	Description
0x00	SPI_TXDATA	R/W	Transmit data register.
0x04	SPI_RXDATA	R	Received data register.
0x08	SPI_CTRL	R/W	bit0 EN; bit1 CPOL; bit2 CPHA.
0x0C	SPI_STATUS	R	bit0 BUSY; bit1 DONE.
0x10	SPI_CLKDIV	R/W	Serial-clock divider/configuration value.

I2C Registers (0x0002_0400-0x0002_04FF)

Offset	Register	Access	Description
0x00	I2C_TXDATA	R/W	Transmit byte/data register.
0x04	I2C_RXDATA	R	Received byte/data register.
0x08	I2C_CTRL	R/W	bit0 START; bit1 STOP; bit2 RW.
0x0C	I2C_STATUS	R	bit0 BUSY; bit1 ACK; bit2 DONE.
0x10	I2C_ADDR	R/W	7-bit target address.

PWM Registers (0x0002_0500-0x0002_05FF)

Offset	Register	Access	Description
0x00	PWM_PERIOD	R/W	Waveform period value.
0x04	PWM_DUTY	R/W	Duty-cycle comparison threshold.
0x08	PWM_CTRL	R/W	bit0 ENABLE; bit1 POLARITY.

4.2 VeeR Machine Interrupt CSRs

CSR	Address	Key field	Description
mie	0x304	bit11 meie	Machine external interrupt enable.
mip	0x344	bit11 meip	Machine external interrupt pending.
mcause	0x342	0x8000_000B	Machine external interrupt cause.

4.3 VeeR PIC Register Set

Register	Purpose
mpiccfg	PIC configuration including priority ordering behaviour.
meiplS	Priority for source S.
meieS	Enable for source S.
meigwctrlS	Gateway control for source S.
meigwclrS	Gateway clear for source S.
meipt	Priority threshold.
meicurpl	Current interrupt priority level.
meicidpl	Current interrupt source/priority state.
meicpct	Interrupt claim/selection state.
meihap	Interrupt handler address/dispatch support.
meivt	External interrupt vector table base.
meipX	Pending information for sources.

4.4 VeeR Debug Module Register Set

DMI offset	Register	Purpose
0x04	data0	Abstract-command data.
0x05	data1	Abstract-command data.
0x10	dmcontrol	Hart/debug control including haltreq, resumereq, ndmreset, dmactive.
0x11	dmstatus	Debug and hart status.
0x16	abstractcs	Abstract-command status/error.
0x17	command	Abstract command selection.
0x18	abstractauto	Abstract-command auto-execution control.
0x38	sbcS	Debug system-bus access control/status.
0x39	sbaddress0	Debug system-bus address.
0x3C	sbdata0	Debug system-bus data lower portion.
0x3D	sbdata1	Debug system-bus data upper portion.
0x40	haltsum0	Hart halt summary.

4.5 Register Design Rule

SCOPE NOTE: No undocumented register, field, channel, interrupt bit, shadow register, buffer, alternate mode, or device-specific command space shall be created merely because an IP implementation could support it. Such an addition is an architectural change and must be explicitly approved.

5. DETAILED CONTROL PATHS AND DATA PATHS - BLOCK BY BLOCK

This section is the implementation-level path specification. Every block uses the same signal names already fixed in the signal list and architecture sections. The purpose is to make the direction of control, address, write-data, read-data, completion and interrupt information unambiguous to Kiro. The descriptions below are intentionally constrained to the project blocks and the selected VeeR EL2 architecture; they do not add new buses, peripherals, registers or protocol modes.

For each block, the control path answers: which event or request causes the block to act, what control signals are generated, and how the operation completes. The data path answers: what information is being moved, in which direction, through which registers/interfaces, and where the information finally goes. Where an exact internal VeeR signal is implementation-specific, the specification deliberately refers to the selected VeeR RTL rather than inventing a new signal name.

5.1 BLOCK 1 - VeeR EL2 PROCESSOR CORE

Purpose

The central RV32IMC, machine-mode-only VeeR EL2 processor. It owns instruction execution, register/CSR state, normal load/store generation, interrupt entry and debug state.

Control Path - step by step

RESET/BOOT:

External/core reset -> rst_vec[31:1] -> PC initialization -> IFU instruction request -> IMEM response -> DEC -> EXU/LSU -> architectural retirement.

NORMAL DATA ACCESS:

Decoded load/store -> LSU control generation -> lsu_haddr + lsu_hwrite + lsu_hsize + lsu_htrans + lsu_hprot -> LSU system-bus interface -> System Interconnect.

COMPLETION:

Selected target returns ready/response -> response path -> lsu_hready / lsu_hresp -> LSU -> instruction completes or exception/next-state handling.

INTERRUPT:

Peripheral IRQ -> VeeR interrupt architecture/PIC as configured -> external interrupt condition -> CPU interrupt entry -> mcause/mepc/handler flow.

DEBUG:

JTAG/DTM/DMI -> Debug Module -> halt/resume/debug operation -> VeeR hart.

Data Path - step by step

INSTRUCTION:

IFU address -> IFU bus -> System Interconnect -> IMEM -> instruction data -> IFU -> DEC.

DATA WRITE:

CPU register operand -> LSU -> lsu_hwdata[63:0] -> System Interconnect -> selected target / Bus Adapter -> peripheral PWDATA or DMEM write data.

DATA READ:

Selected target read data -> Bus Adapter/response mux -> lsu_hrdata[63:0] -> LSU -> CPU register writeback.

CONTROL/CSR:

Decoded CSR operation -> VeeR CSR state -> control response to the core. Peripheral registers are not implemented as CPU CSRs.

Signals and ownership

Signal / group	Role
clk	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
rst_l	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
rst_vec[31:1]	Use exactly this interface/control name; internal VeeR-

	only details follow selected RTL.
Isu_haddr	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
Isu_hwdata	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
Isu_hwrite	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
Isu_hsize	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
Isu_htrans	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
Isu_hprot	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
Isu_hrdata	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
Isu_hready	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
Isu_hresp	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
interrupt inputs	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
debug interface	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
core_id[31:4]	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

Clock/reset inputs -> block-specific timing/control -> downstream functional blocks

Reset, interrupt and scope boundary

Reset follows the selected VeeR EL2 configuration. External peripheral interrupts use the VeeR external-interrupt architecture; NMI remains separate. Do not replace VeeR or embed project peripheral functions in the processor.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.2 BLOCK 2 - IFU

Purpose

Instruction Fetch Unit. It owns the instruction-side request/response path and supplies instruction words to the decode stage.

Control Path - step by step

PC/RESET -> instruction request -> IFU AHB-Lite control -> target address decode -> IMEM access -> wait/response -> instruction accepted -> PC update/next fetch.

The IFU may create instruction-side speculative accesses supported by the selected VeeR implementation; these remain instruction-side transactions and must not be converted into LSU peripheral writes.

Data Path - step by step

IFU address -> haddr -> System Interconnect -> imem_sel -> IMEM -> instruction read data -> response mux/IFU -> instruction word.

No peripheral application data travels through the IFU path.

Signals and ownership

Signal / group	Role
haddr	Use exactly this interface/control name; internal VeeR-

	only details follow selected RTL.
hburst	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hmastlock	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hprot	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hsize	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
htrans	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hwrite	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hrdata	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hready	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hresp	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
ifu_bus_clk_en	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

PC/address -> IFU request -> AHB-Lite -> System Interconnect -> IMEM -> instruction response
-> IFU -> DEC

Reset, interrupt and scope boundary

Reset returns the block to its selected VeeR-defined/project-defined state. No new project-specific protocol or side path is introduced.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.3 BLOCK 3 - DEC

Purpose

Instruction decode and control generation stage. It interprets the fetched RISC-V instruction and determines the required execution, LSU, CSR and control operations.

Control Path - step by step

Instruction word -> opcode/funct/register/immediate decode -> control fields -> EXU or LSU or CSR control -> next pipeline action.

Data Path - step by step

Fetched instruction bits -> decode fields/immediate -> execution operands/control. The decode stage does not transform application payload data and does not decode peripheral addresses.

Signals and ownership

Signal / group	Role
internal VeeR decode fields	Selected VeeR internal control set.
execution control	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
LSU control	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
CSR control	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

Instruction/control input -> decode/execute control -> result or LSU/CSR action -> next pipeline state

Reset, interrupt and scope boundary

Reset returns the block to its selected VeeR-defined/project-defined state. No new project-specific protocol or side path is introduced.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.4 BLOCK 4 - EXU

Purpose

Execution Unit. Performs arithmetic, logical, branch, comparison and other execution-stage operations supported by the selected VeeR EL2 configuration.

Control Path - step by step

Decoded execute control -> operand selection -> ALU/branch/other selected execution logic -> result/branch decision -> writeback or LSU/CSR control.

Data Path - step by step

Register operands + immediate -> execution datapath -> result -> register writeback, branch decision, or address/control input for LSU.

Signals and ownership

Signal / group	Role
internal VeeR EXU signals	Selected VeeR internal control set.

Transaction example

Instruction/control input -> decode/execute control -> result or LSU/CSR action -> next pipeline state

Reset, interrupt and scope boundary

Reset returns the block to its selected VeeR-defined/project-defined state. No new project-specific protocol or side path is introduced.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.5 BLOCK 5 - LSU

Purpose

Load/Store Unit. Generates the processor data-side transactions used for DMEM and all memory-mapped project peripherals.

Control Path - step by step

Decoded load/store -> access size/type/address/control -> lsu_haddr + lsu_hwrite + lsu_hsize + lsu_htrans + lsu_hprot -> system-bus request -> selected target -> lsu_hready/lsu_hresp -> load writeback or store completion.

Data Path - step by step

STORE: CPU register data -> lsu_hwdata[63:0] -> system interconnect -> selected target.

LOAD: selected target data -> response mux -> lsu_hrdata[63:0] -> LSU -> CPU register writeback.

The LSU does not directly create PSEL/PENABLE; the Bus Adapter owns the processor-to-peripheral protocol conversion.

Signals and ownership

Signal / group	Role
lsu_haddr	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hburst	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hmastlock	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hprot	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hsize	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_htrans	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hwdata	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hwrite	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hrdata	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hready	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hresp	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

CPU load/store -> LSU -> lsu_haddr/lsu_hwdata/lsu_hwrite -> target decode -> target access -> lsu_hready/lsu_hresp -> CPU

Reset, interrupt and scope boundary

Reset returns the block to its selected VeeR-defined/project-defined state. No new project-specific protocol or side path is introduced.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.6 BLOCK 6 - IFU AHB-LITE MASTER INTERFACE

Purpose

Processor instruction-side system-bus boundary.

Control Path - step by step

IFU request -> AHB-Lite address/control valid -> target selection -> wait/ready -> instruction response.

The interface remains instruction-side and uses ifu_bus_clk_en according to the selected VeeR configuration.

Data Path - step by step

Instruction address is presented on the IFU AHB-Lite interface. Returned instruction data appears on hrdata and is consumed by the IFU. No LSU application-data path is merged here.

Signals and ownership

Signal / group	Role
haddr	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hburst	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hmastlock	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hprot	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hsize	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
htrans	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hwrite	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hrdata	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hready	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
hresp	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
ifu_bus_clk_en	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

Clock/reset inputs -> block-specific timing/control -> downstream functional blocks

Reset, interrupt and scope boundary

Reset returns the block to its selected VeeR-defined/project-defined state. No new project-specific protocol or side path is introduced.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.7 BLOCK 7 - LSU AHB-LITE MASTER INTERFACE

Purpose

Processor data-side system-bus boundary for DMEM and memory-mapped I/O.

Control Path - step by step

LSU request -> AHB-Lite address/control -> System Interconnect -> target -> response/ready -> LSU.

The interface must preserve the selected VeeR transfer semantics and must not bypass the System Interconnect.

Data Path - step by step

Write path: lsu_hwdata -> selected target write data.

Read path: selected target read data -> lsu_hrdata.

Control/response signals determine whether the transaction completed successfully.

Signals and ownership

Signal / group	Role
lsu_haddr	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hburst	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

lsu_hmastlock	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hprot	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hsize	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_htrans	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hwdata	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hwrite	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hrdata	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hready	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hresp	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_bus_clk_en	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

CPU load/store -> LSU -> lsu_haddr/lsu_hwdata/lsu_hwrite -> target decode -> target access -> lsu_hready/lsu_hresp -> CPU

Reset, interrupt and scope boundary

Reset returns the block to its selected VeeR-defined/project-defined state. No new project-specific protocol or side path is introduced.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.8 BLOCK 8 - SYSTEM INTERCONNECT

Purpose

Central routing fabric between VeeR master interfaces and project memories/peripherals.

Control Path - step by step

MASTER REQUEST -> capture active request -> decode system address -> generate exactly one target select -> forward control -> wait for selected target -> capture response -> route response to the requesting master.

Data Path - step by step

Forward: master write data -> selected target.

Return: selected target read data -> response mux -> requesting master.

No application data is interpreted; only routing/selection/response metadata is handled.

Signals and ownership

Signal / group	Role
imem_sel	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
dmem_sel	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
psel[5:0]	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
master request/response signals	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
target response signals	Use exactly this interface/control name; internal VeeR-

	only details follow selected RTL.
--	-----------------------------------

Transaction example

Clock/reset inputs -> block-specific timing/control -> downstream functional blocks

Reset, interrupt and scope boundary

Reset returns routing/adaptor/bus state to idle. No stale transaction may survive reset. PSEL/PENABLE remain inactive during peripheral reset.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.9 BLOCK 9 - ADDRESS DECODER

Purpose

Maps the fixed 32-bit project address map to exactly one memory or peripheral destination.

Control Path - step by step

System address -> range comparison -> one-hot target selection.

```
0x0000_0000-0x0000_FFFF -> imem_sel.
0x0001_0000-0x0001_FFFF -> dmem_sel.
0x0002_0000-0x0002_00FF -> psel[0].
0x0002_0100-0x0002_01FF -> psel[1].
0x0002_0200-0x0002_02FF -> psel[2].
0x0002_0300-0x0002_03FF -> psel[3].
0x0002_0400-0x0002_04FF -> psel[4].
0x0002_0500-0x0002_05FF -> psel[5].
0x0002_0600-0xFFFF_FFFF -> reserved/unmapped.
```

Data Path - step by step

The decoder does not alter application payload data. It transforms the address into selection metadata. A valid access must not activate two targets simultaneously.

Signals and ownership

Signal / group	Role
32-bit system address	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
imem_sel	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
dmem_sel	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
psel[5:0]	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

Instruction/control input -> decode/execute control -> result or LSU/CSR action -> next pipeline state

Reset, interrupt and scope boundary

Reset returns routing/adaptor/bus state to idle. No stale transaction may survive reset. PSEL/PENABLE remain inactive during peripheral reset.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.10 BLOCK 10 - RESPONSE MUX

Purpose

Selects the response from the target that owns the active transaction and returns it to the corresponding VeeR master interface.

Control Path - step by step

Active target select -> choose corresponding read-data/ready/response -> forward only that response -> deassert/invalidate non-selected responses.

Data Path - step by step

IMEM/DMEM/memory response or peripheral PRDATA -> selected response -> hrdata/lsu_hrdata as appropriate.

No unselected target data may reach the CPU-side response bus.

Signals and ownership

Signal / group	Role
target read data	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
target ready	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
target response	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hrdata	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hready	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_hresp	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
IFU hrdata/hready/hresp where applicable	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

Selected target -> read data + ready + response -> CPU-side response bus

Reset, interrupt and scope boundary

Reset returns routing/adaptor/bus state to idle. No stale transaction may survive reset. PSEL/PENABLE remain inactive during peripheral reset.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.11 BLOCK 11 - AHB-LITE TO APB-STYLE BUS ADAPTER

Purpose

Protocol and width adaptation layer between the VeeR selected processor-side transaction and the six project APB-style peripherals.

Control Path - step by step

AHB-Lite request -> hold address/control/data -> APB SETUP: PSEL=1, PENABLE=0 -> APB ACCESS: PENABLE=1 -> wait while PREADY=0 -> capture PRDATA/PREADY/response -> translate completion back to the processor-side bus.

Data Path - step by step

Write: lsu_hwdata[63:0] -> selected 32-bit PWDATA transaction as required by access size.

Read: PRDATA[31:0] -> response formatting -> lsu_hrdata[63:0].

The adapter owns PADDR/PSEL/PENABLE/PWRITE timing; peripherals do not see raw lsu_* signals.

Signals and ownership

Signal / group	Role
PCLK	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PRESETn	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PADDR[31:0]	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PSEL	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PENABLE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWRITE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWDATA[31:0]	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PRDATA[31:0]	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PREADY	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
processor-side lsu_* signals	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

Processor transaction -> SETUP -> ACCESS -> wait while PREADY=0 -> COMPLETE -> response to processor

Reset, interrupt and scope boundary

Reset returns routing/adaptor/bus state to idle. No stale transaction may survive reset. PSEL/PENABLE remain inactive during peripheral reset.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.12 BLOCK 12 - APB-STYLE PERIPHERAL BUS

Purpose

Common memory-mapped register transport shared by UART, Timer, GPIO, SPI, I2C and PWM.

Control Path - step by step

SETUP -> PSEL=1, PENABLE=0.

ACCESS -> PSEL=1, PENABLE=1.

WAIT -> hold address/control/data while PREADY=0.

COMPLETE -> PREADY=1; response/data returned.

Data Path - step by step

PWDATA -> selected peripheral register write.

Selected peripheral PRDATA -> adaptor -> CPU-side read data.

The bus itself carries register access information, not serial protocol data such as MOSI or SDA.

Signals and ownership

Signal / group	Role
PCLK	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PRESETn	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PADDR[31:0]	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PSEL	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PENABLE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWRITE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWDATA[31:0]	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PRDATA[31:0]	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PREADY	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

Processor transaction -> SETUP -> ACCESS -> wait while PREADY=0 -> COMPLETE -> response to processor

Reset, interrupt and scope boundary

Reset returns routing/adaptor/bus state to idle. No stale transaction may survive reset. PSEL/PENABLE remain inactive during peripheral reset.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.13 BLOCK 13 - INSTRUCTION MEMORY (IMEM)

Purpose

64 KB project instruction memory occupying 0x0000_0000-0x0000_FFFF.

Control Path - step by step

IFU request -> address decode -> imem_sel -> IMEM access enable/read -> memory response -> IFU.

For instruction access, the memory interface must complete every valid request.

Data Path - step by step

Instruction address -> memory array lookup -> instruction read data -> IFU.

The memory stores the boot/program image; it is not automatically converted into VeeR ICCM or I-cache.

Signals and ownership

Signal / group	Role
instruction address	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
imem_sel	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
instruction read data	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
ready/response	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

IFU address -> IMEM -> instruction data -> IFU

Reset, interrupt and scope boundary

Reset returns the block to its selected VeeR-defined/project-defined state. No new project-specific protocol or side path is introduced.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.14 BLOCK 14 - DATA MEMORY (DMEM)

Purpose

64 KB project read/write data memory occupying 0x0001_0000-0x0001_FFFF.

Control Path - step by step

LSU request -> address decode -> dmem_sel -> read/write control -> DMEM operation -> completion -> LSU.

Data Path - step by step

Write: lsu_hwdata -> DMEM selected byte/word lanes according to access size.

Read: DMEM -> response path -> lsu_hrdata.

DMEM is distinct from VeeR optional DCCM.

Signals and ownership

Signal / group	Role
data address	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
write data	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
read data	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
read/write control	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
byte/transfer-valid	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
dmem_sel	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
ready/response	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

LSU store -> DMEM write; LSU load -> DMEM read -> response

Reset, interrupt and scope boundary

Reset returns the block to its selected VeeR-defined/project-defined state. No new project-specific protocol or side path is introduced.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.15 BLOCK 15 - UART CONTROLLER

Purpose

Asynchronous serial communication peripheral with TX/RX and software-visible control/status/data registers.

Control Path - step by step

CPU register access -> address decode -> psel[0] -> Bus Adapter -> UART register decoder -> register control/status -> serial state machine.
RX event -> UART_RX_IRQ -> PIC/external interrupt path.

Data Path - step by step

TX: CPU -> lsu_hwdata -> PWDATA -> UART_TXDATA -> TX serializer -> TX pin.
RX: RX pin -> RX sampler/receiver -> UART_RXDATA -> PRDATA -> CPU.
STATUS readback reports the defined TX/RX state.

Signals and ownership

Signal / group	Role
PADDR	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PSEL	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PENABLE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWRITE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PRDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PREADY	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
TX	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
RX	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
UART_RX_IRQ	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
UART_TXDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
UART_RXDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
UART_STATUS	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
UART_BAUD	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

CPU -> UART_TXDATA -> TX; RX -> UART_RXDATA -> CPU; RX event -> UART_RX_IRQ

Reset, interrupt and scope boundary

PRESETn resets the peripheral register/control state. UART_RX_IRQ or TMR_IRQ is generated only for the defined event.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.16 BLOCK 16 - TIMER CONTROLLER

Purpose

Programmable timing/compare peripheral that generates TMR_IRQ for its defined timer event.

Control Path - step by step

CPU writes TMR_CTRL/TMR_COMPARE -> psel[1] -> Timer registers updated -> counter enabled/runs -> compare evaluation -> timer event -> TMR_IRQ.
Register reads use the normal APB path.

Data Path - step by step

TMR_COMPARE supplies the compare threshold. TMR_COUNT represents current count/state. Timer logic compares current count against the programmed condition. Register readback returns through PRDATA.

Signals and ownership

Signal / group	Role
PADDR	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PSEL	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PENABLE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWRITE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PRDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PREADY	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
TMR_COUNT	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
TMR_COMPARE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
TMR_CTRL	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
TMR_IRQ	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

CPU -> TMR_CTRL/TMR_COMPARE -> counter -> compare -> TMR_IRQ

Reset, interrupt and scope boundary

PRESETn resets the peripheral register/control state. UART_RX_IRQ or TMR_IRQ is generated only for the defined event.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.17 BLOCK 17 - GPIO CONTROLLER

Purpose

Eight general-purpose digital input/output pins controlled and observed through memory-mapped registers.

Control Path - step by step

CPU access -> psel[2] -> GPIO register decoder -> GPIO_DIR / GPIO_OUT control. Input sampling is continuous/clocked according to the selected RTL and feeds GPIO_IN.

Data Path - step by step

Write: PWDATA -> GPIO_OUT / GPIO_DIR -> output/direction logic -> gpio_out[7:0].

Read: gpio_in[7:0] -> GPIO_IN -> PRDATA -> CPU.

GPIO does not generate a project interrupt in the baseline architecture.

Signals and ownership

Signal / group	Role
PADDR	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PSEL	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PENABLE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWRITE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PRDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PREADY	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
GPIO_OUT	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
GPIO_IN	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
GPIO_DIR	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
gpio_in[7:0]	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
gpio_out[7:0]	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

CPU -> GPIO_DIR/GPIO_OUT -> pins; pins -> GPIO_IN -> CPU

Reset, interrupt and scope boundary

PRESETn resets control/register state. GPIO and PWM do not generate baseline project interrupt sources.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.18 BLOCK 18 - SPI CONTROLLER

Purpose

Full-duplex synchronous serial master. It transmits on MOSI, receives on MISO, generates SCLK and controls CS_n. It raises SPI_DONE_IRQ for the defined transfer-done event.

Control Path - step by step

CPU writes SPI_TXDATA/SPI_CTRL -> psel[3] -> register decoder -> transfer FSM -> CS_n assert -> SCLK generation -> TX/RX shifting -> final bit -> transfer-done state -> SPI_DONE_IRQ.

CPU register transaction completion (PREADY) is separate from external serial transfer completion.

Data Path - step by step

TX: lsu_hwdata -> PWDATA -> SPI_TXDATA -> TX shift register -> MOSI.

RX: MISO -> RX shift register -> SPI_RXDATA -> PRDATA -> CPU.

SCLK timing is generated from the SPI timing/control logic using PCLK as the local controller clock.

Signals and ownership

Signal / group	Role
PADDR	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PSEL	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PENABLE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWRITE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PRDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PREADY	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
SCLK	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
MOSI	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
MISO	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
CS_n	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
SPI_CTRL	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
SPI_STATUS	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
SPI_DONE_IRQ	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
SPI_TXDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
SPI_RXDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

CPU -> SPI_TXDATA/CTRL -> FSM -> CS_n/SCLK/MOSI/MISO -> SPI_DONE_IRQ -> CPU

Reset, interrupt and scope boundary

PRESETn returns the controller to its defined reset state and aborts an active serial transaction. The serial transfer completion event is separate from APB register-access completion.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.19 BLOCK 19 - I2C CONTROLLER

Purpose

Standard-mode two-wire I2C master with 7-bit addressing, byte-level read/write, START/STOP generation and ACK/NACK handling.

Control Path - step by step

CPU writes/configures -> psel[4] -> I2C register decoder -> I2C FSM -> START -> 7-bit address + R/W -> ACK/NACK -> byte data phase -> ACK/NACK -> STOP -> transfer done -> I2C_DONE_IRQ.

Data Path - step by step

TX: I2C_TXDATA -> TX shift/control -> SDA.

RX: SDA -> input sampler -> RX shift/register -> I2C_RXDATA -> PRDATA -> CPU.

I2C_ADDR carries the external 7-bit target address; it is not the same as the SoC PADDR.

Signals and ownership

Signal / group	Role
PADDR	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PSEL	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PENABLE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWRITE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PRDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PREADY	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
SDA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
SCL	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
I2C_CTRL	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
I2C_STATUS	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
I2C_ADDR	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
I2C_TXDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
I2C_RXDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
I2C_DONE_IRQ	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

CPU -> I2C control/data -> START/address/ACK/data/STOP -> I2C_DONE_IRQ -> CPU; SDA -> RXDATA on reads

Reset, interrupt and scope boundary

PRESETn returns the controller to its defined reset state and aborts an active serial transaction. The serial transfer completion event is separate from APB register-access completion.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.20 BLOCK 20 - PWM GENERATOR

Purpose

Custom in-house programmable waveform generator using a free-running counter, period register, duty-cycle compare register and comparator.

Control Path - step by step

CPU writes PWM_PERIOD/PWM_DUTY/PWM_CTRL -> psel[5] -> PWM register decoder -> configuration state -> counter/comparator operation -> PWM_OUT.

Data Path - step by step

PCLK -> free-running counter.
PWM_PERIOD controls the counter cycle.
PWM_DUTY provides the compare threshold.
Comparator evaluates counter versus duty -> PWM_OUT.
CPU readback uses PRDATA through the adapter.

Signals and ownership

Signal / group	Role
PADDR	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PSEL	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PENABLE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWRITE	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PRDATA	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PREADY	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWM_PERIOD	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWM_DUTY	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWM_CTRL	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PWM_OUT	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

CPU -> PWM_PERIOD/PWM_DUTY/PWM_CTRL -> counter/comparator -> PWM_OUT

Reset, interrupt and scope boundary

PRESETn resets control/register state. GPIO and PWM do not generate baseline project interrupt sources.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.21 BLOCK 21 - VeeR PIC

Purpose

Core-integrated programmable interrupt controller for external interrupt sources. It provides gateway synchronization, source enable, priority and threshold evaluation and selects the winning source for the VeeR external interrupt path.

Control Path - step by step

Peripheral IRQ -> exintsrc_req[source] -> gateway/synchronizer -> pending state -> source enable check -> priority vs threshold -> priority comparison -> winning source ID -> PIC target -> VeeR external interrupt -> CPU ISR.

Data Path - step by step

The PIC data is interrupt state rather than application payload: pending bits, source ID, priority, enable state, claim/vector information. Peripheral payload data remains on each peripheral's normal LSU/APB path.

Signals and ownership

Signal / group	Role
TMR_IRQ	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
UART_RX_IRQ	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
SPI_DONE_IRQ	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
I2C_DONE_IRQ	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
mpiccfg	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
meieS	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
meipIS	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
meipt	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
meicpct	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
meihap	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
meivt	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
meipX	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
gateway signals	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

Peripheral IRQ -> gateway -> pending -> enable/priority/threshold -> winner -> VeeR external interrupt

Reset, interrupt and scope boundary

Reset returns the block to its selected VeeR-defined/project-defined state. No new project-specific protocol or side path is introduced.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.22 BLOCK 22 - DEBUG MODULE AND JTAG

Purpose

VeeR core-complex debug infrastructure for external debugger transport, hart halt/resume, abstract commands and supported debug system-bus memory access.

Control Path - step by step

Debugger -> JTAG -> DTM -> DMI -> Debug Module -> dmcontrol/command/sb* operation -> hart control or Debug Bus Master -> result/status -> DMI -> JTAG -> debugger.

Data Path - step by step

Debug register data: DMI -> DM register -> DMI response.

System-bus memory data: sbaddress0/sbdata0/sbdata1 -> Debug Bus Master -> supported SoC memory -> response -> sbdata* -> DMI.

Abstract command data uses data0/data1 and the selected debug command mechanism.

Signals and ownership

Signal / group	Role
dbg_rst_l	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
dbg_bus_clk_en	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
JTAG/DTM/DMI signals	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
sb_h* debug bus signals	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
dmcontrol	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
dmstatus	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
abstractcs	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
command	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
abstractauto	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
data0	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
data1	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
sbc	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
sbaddress0	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
sbdata0	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
sbdata1	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
haltsum0	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

Debugger -> JTAG -> DTM -> DMI -> DM -> halt/resume or debug-bus operation -> DMI -> debugger

Reset, interrupt and scope boundary

rst_l resets the VeeR core complex but excludes the Debug Module; dbg_rst_l resets the Debug Module/JTAG synchronizers. ndmreset is a Debug Module control for core reset while leaving the Debug Module active.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

5.23 BLOCK 23 - CLOCK AND RESET CONNECTIVITY

Purpose

System timing/reset infrastructure connecting the VeeR core complex, project peripheral bus and Debug Module.

Control Path - step by step

CLOCK: clock source/distribution -> clk for VeeR -> PCLK for project peripherals as defined by integration.

RESET: external/system reset -> rst_l for VeeR core complex; PRESETn for project peripherals; dbg_rst_l for Debug Module/JTAG synchronization reset.

BOOT: stable clock -> reset held asserted -> reset released -> rst_vec[31:1] / project boot mapping -> IFU starts fetching.

Data Path - step by step

Clock carries timing, not application payload data.

Reset establishes known state.

VeeR bus clock enables govern interface timing relationships: ifu_bus_clk_en, lsu_bus_clk_en, dbg_bus_clk_en, dma_bus_clk_en.

Signals and ownership

Signal / group	Role
clk	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PCLK	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
rst_l	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
PRESETn	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
dbg_rst_l	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
rst_vec[31:1]	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
ifu_bus_clk_en	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
lsu_bus_clk_en	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
dbg_bus_clk_en	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.
dma_bus_clk_en	Use exactly this interface/control name; internal VeeR-only details follow selected RTL.

Transaction example

Clock/reset inputs -> block-specific timing/control -> downstream functional blocks

Reset, interrupt and scope boundary

Clock must be stable before VeeR core reset release. Peripheral reset uses PRESETn. SPI and I2C active transfers are aborted by peripheral reset according to the project reset rules. Do not add undocumented PLLs, clocks or reset domains.

SCOPE LOCK: This block shall perform only the responsibilities defined above. Any requested feature that adds a new interface, new register, new address range, new interrupt source, new clock/reset domain, or new peripheral function must be treated as an explicit scope change.

6. COMPLETE END-TO-END CONTROL AND DATA PATHS

6.1 Instruction Path

Reset vector → VeeR IFU → IFU AHB-Lite → System Interconnect → IMEM
IMEM read data → IFU → DEC → EXU → retire

6.2 Data Memory Load

CPU load → DEC/LSU → lsu_haddr/lsu_hsize/lsu_htrans → System Interconnect
→ Address Decoder → dmem_sel → DMEM → read data → Response MUX → lsu_hrdata → LSU → CPU

6.3 Data Memory Store

CPU store → LSU → lsu_haddr/lsu_hwdata/lsu_hwrite → System Interconnect
→ Address Decoder → dmem_sel → DMEM → completion → lsu_hready/lsu_hresp

6.4 Peripheral Write

CPU store → LSU → lsu_haddr/lsu_hwdata/lsu_hwrite → System Interconnect
→ Address Decoder → one psel[n] → Bus Adapter → PADDR/PWRITE/PWDATA/PSEL/PENABLE
→ Peripheral register → Peripheral function → completion PREADY → Adapter → lsu_hready

6.5 Peripheral Read

CPU load → LSU → lsu_haddr → System Interconnect → Address Decoder → one psel[n]
→ Bus Adapter → PADDR/PSEL/PENABLE/PWRITE=0 → Peripheral register → PRDATA
→ Bus Adapter → Response MUX → lsu_hrdata → LSU → CPU

6.6 Interrupt Path

Peripheral event → peripheral IRQ → VeeR PIC gateway/synchronizer → pending
→ source enable → priority/threshold → winning source → VeeR external interrupt
→ mie[11]/mip[11]/mcause class → ISR → peripheral STATUS/data access through LSU

6.7 Debug Path

Debugger → JTAG → DTM → DMI → Debug Module
→ dmcontrol/abstract command/debug system-bus operation
→ Hart control OR Debug Bus Master → supported SoC memory
→ DMI → JTAG → debugger

6.8 Unmapped Access

CPU transaction → LSU → System Interconnect → Address Decoder → no target
→ defined unmapped response → PRDATA=0x0000_0000 on APB-style side
→ PREADY after one wait cycle → processor-side response → CPU

6.9 Side-Effecting I/O Rule

The interconnect and adapter must preserve the single intended peripheral access. They shall not duplicate, merge, reorder, or split side-effecting register transactions in a way that changes peripheral behaviour unless such behaviour is explicitly defined by the selected bus implementation.

7. NON-FATAL ERROR AND EXCEPTION-BOUNDARY SPECIFICATION

7.1 Philosophy

The project requires all non-fatal error/abnormal conditions to be visible at the correct architectural level. A peripheral protocol condition is not automatically a CPU exception, and a Debug Module protocol error is not automatically a processor NMI.

7.2 System-Level Non-Fatal Conditions

Condition	Detection/Indicator	Required behaviour
Unmapped system address	No target select	Return defined response; do not leave transaction outstanding.
Undefined peripheral register offset	Peripheral register decoder	Return defined local/bus response; do not invent a register.
Unsupported access format	Bus Adapter/interconnect	Return defined error/response without hanging the bus.
Peripheral wait state	PREADY=0	Hold ACCESS transaction stable until completion.
Reset during active SPI transfer	PRESETn	Abort SPI transfer and clear active state.
Reset during active I2C transfer	PRESETn	Abort I2C transfer and clear active state.
Core reset during active SoC memory access	rst_l	Follow VeeR documented reset restriction; do not claim guaranteed recovery if the implementation does not provide it.

7.3 Peripheral Conditions

Block	Condition	Architectural level	Not to be reinterpreted as
UART	TX_BUSY/RX_VALID	UART_STATUS	NMI
Timer	Compare match	TMR_IRQ/TMR_STATUS	VeeR timer interrupt unless explicitly connected that way
SPI	BUSY/DONE	SPI_STATUS/SPI_DONE_IRQ	NMI
I2C	ACK/NACK/BUSY/DONE	I2C_STATUS/I2C_DONE_IRQ	NMI
GPIO	Digital input/output state	GPIO_IN/GPIO_OUT/ GPIO_DIR	GPIO IRQ in baseline
PWM	Waveform configuration/output	PWM_registers/PWM_OUT	PWM IRQ in baseline

7.4 Debug Non-Fatal Errors

Condition	Indicator	Meaning
Debug system bus already busy	sbbusy/sbbusyerror	Overlapping access attempted.
Unsupported Debug system-bus access size	sbberror	No access performed for unsupported size; error reported.
Abstract command problem	abstractcs.cmderr	Command execution error/status.
Invalid/unimplemented control field	Register-specific result	Operation follows VeeR implementation; do not invent unsupported function.

7.5 Processor-Level NMI Boundary

The VeeR NMI mechanism is reserved for processor-level serious error conditions documented by VeeR. Project peripheral conditions such as I2C NACK, SPI BUSY, UART RX status and Timer compare remain ordinary peripheral status/interrupt conditions.

8. COMPLETE INTERRUPT SPECIFICATION

8.1 Baseline Project Sources

IRQ	Source event	Enable/status owner	CPU path
TMR_IRQ	Timer compare event	Timer TMR_CTRL/status	Timer → PIC/external IRQ → CPU
UART_RX_IRQ	UART receive event	UART status	UART → PIC/external IRQ → CPU
SPI_DONE_IRQ	SPI transfer complete	SPI_STATUS	SPI → PIC/external IRQ → CPU
I2C_DONE_IRQ	I2C transfer complete	I2C_STATUS	I2C → PIC/external IRQ → CPU

8.2 No Additional Project IRQs

- No GPIO interrupt in baseline.
- No PWM interrupt in baseline.
- No IMEM/DMEM interrupt.
- No interconnect interrupt.
- No second global PLIC.

8.3 PIC Decision Chain

IRQ source → gateway/synchronizer → pending → enabled? → priority vs threshold
→ compare eligible sources → winning source ID → external interrupt target → CPU ISR

8.4 Machine-Level Relationship

Peripheral IRQ → VeeR external interrupt subsystem → machine external interrupt
mie[11]=meie; mip[11]=meip; mcause class = 0x8000_000B

8.5 Interrupt Service Rule

The ISR shall use the VeeR source identification mechanism where configured and read the source peripheral STATUS register to determine the detailed event. The peripheral condition is then serviced/cleared using that peripheral's defined register mechanism. The PIC does not transport peripheral payload data.

9. CLOCK, RESET, BOOT AND INITIALIZATION

9.1 Clock Rules

- VeeR core-complex clock is clk.
- Project peripheral bus clock is PCLK.
- Their relationship must be explicitly defined; a common synchronous clock is permitted for a simple implementation when consistent with the selected VeeR configuration.
- Do not add independent clocks without an explicit architecture decision.
- Use ifu_bus_clk_en, lsu_bus_clk_en, dbg_bus_clk_en and dma_bus_clk_en only according to the selected VeeR interfaces.

9.2 Reset Rules

Signal	Scope
rst_l	VeeR core complex excluding Debug Module
PRESETn	Project peripherals and APB-style interface
dbg_rst_l	Debug Module and JTAG synchronizers

9.3 Boot Sequence

```
Clock stable
↓
Reset asserted
↓
Reset vector configured consistently with 0x0000_0000 IMEM
↓
Core reset released
↓
VeeR fetch begins at project boot address
↓
Program executes
```

9.4 Peripheral Reset

```
PRESETn asserted
→ UART/Timer/GPIO/SPI/I2C/PWM control/status reset
→ PSEL/PENABLE inactive
→ SPI/I2C active transactions aborted
→ peripherals return to defined idle/reset state
```

9.5 Debug Reset

```
dbg_rst_l asserted → Debug Module/JTAG synchronizers reset
ndmreset asserted → VeeR hart/core complex reset while Debug Module remains active
```

10. APPLICATIONS WITHIN PROJECT SCOPE

Application	Blocks used	Allowed path
Software console / monitoring	VeeR + UART	CPU/LSU → UART registers → TX/RX
Periodic scheduling	VeeR + Timer	CPU config → Timer → TMR_IRQ → ISR
Switch/LED digital I/O	VeeR + GPIO	CPU → GPIO_OUT/GPIO_DIR or GPIO_IN
External sensor / flash / display SPI link	VeeR + SPI	CPU → SPI registers → SPI serial pins
Sensor / EEPROM interface	VeeR + I2C	CPU → I2C registers → SDA/SCL
Motor speed control	VeeR + PWM + external driver	CPU → PWM_PERIOD/PWM_DUTY → PWM_OUT
Sensor-driven actuator control	I2C or SPI + VeeR + PWM	Peripheral data → CPU → PWM

Application	Blocks used	Allowed path configuration

SCOPE NOTE: Applications demonstrate combinations of the defined blocks; they do not create new hardware blocks. A motor driver, LED driver, sensor device, EEPROM or display is external application hardware, not part of the corresponding project peripheral controller.

11. VERIFICATION REQUIREMENTS

11.1 Unit-Level Verification

1. Verify every project peripheral register read/write path.
2. Verify each peripheral local address decode and reserved offsets.
3. Verify reset state and idle behaviour.
4. Verify each defined external interface.
5. Verify Timer compare/IRQ behaviour.
6. Verify SPI full-duplex transfer and DONE status/IRQ.
7. Verify I2C START/STOP/address/ACK/NACK/byte transfer behaviour.
8. Verify PWM period/duty waveform generation.
9. Verify GPIO direction and data paths.

11.2 Interconnect Verification

10. Verify IMEM and DMEM address ranges.
11. Verify each psel[0:5] mapping.
12. Verify boundary addresses between every adjacent region.
13. Verify no two selects are active for one valid peripheral access.
14. Verify unmapped access returns a defined response after one wait cycle.
15. Verify read-data response comes only from the selected target.
16. Verify write data reaches only the selected target.
17. Verify every system-bus request receives a completion/error response.

11.3 Interrupt Verification

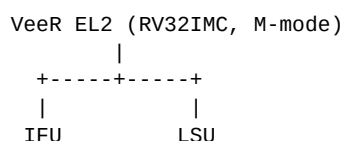
18. Verify each of the four project IRQ sources.
19. Verify level-sensitive behaviour.
20. Verify per-source enable/priority/threshold behaviour when PIC is enabled in the selected VeeR configuration.
21. Verify simultaneous interrupts select the correct winning source according to VeeR PIC rules.
22. Verify ISR can identify and service the source.
23. Verify GPIO/PWM do not accidentally generate project IRQs.

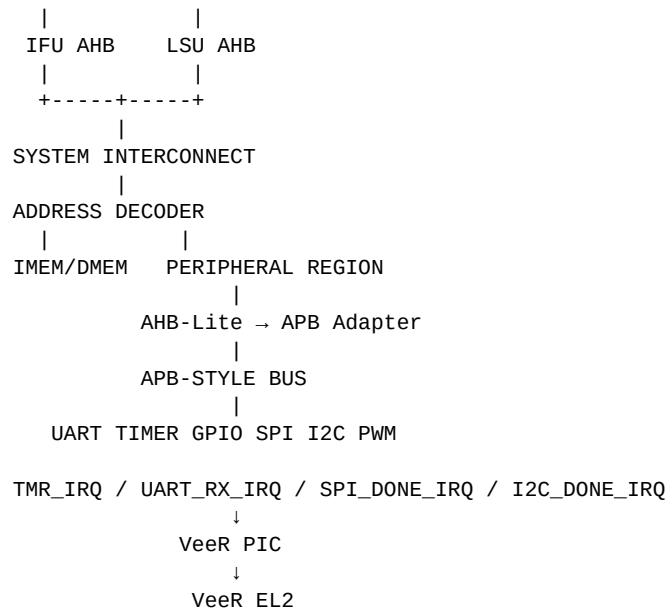
11.4 Reset/Debug Verification

24. Verify clock stable before VeeR reset release.
25. Verify project peripherals reset to known state.
26. Verify SPI/I2C active transfers abort on PRESETn.
27. Verify rst_l and dbg_rst_l have separate scope.
28. Verify debug halt/resume/reset operations according to the selected VeeR implementation.
29. Verify Debug system-bus access uses its dedicated path and does not silently become an APB master.

12. FINAL KIRO IMPLEMENTATION CONTRACT

12.1 Mandatory Architecture





12.2 Mandatory Scope Rules

30. Use VeeR EL2; no substitute processor.
31. Use baseline AHB-Lite processor-side configuration; do not implement AXI4 simultaneously unless the architecture changes.
32. Keep IFU and LSU separate.
33. Keep the exact project address map.
34. Keep the APB-style peripheral signal set unchanged.
35. Implement only UART, Timer, GPIO, SPI, I2C and PWM as project peripherals.
36. Keep only TMR_IRQ, UART_RX_IRQ, SPI_DONE_IRQ and I2C_DONE_IRQ as project peripheral IRQs.
37. No GPIO IRQ or PWM IRQ.
38. No project DMA controller.
39. No external PLIC unless explicitly approved as an architecture change.
40. No additional CPU cores.
41. No additional peripheral instances or buses.
42. No undocumented registers or protocol modes.
43. Every VeeR system-bus request must receive a defined response.
44. Peripheral region must retain side-effecting I/O treatment.
45. All debug, PIC, optional VeeR, clock and reset features must follow the selected VeeR implementation and this scope contract.
46. Any request that adds functionality not present here must be treated as a scope-change request, not inferred implementation detail.

12.3 Explicitly Out of Scope Unless Approved

- AXI4 + AHB-Lite dual processor bus implementation.
- Extra UART/SPI/I2C/GPIO/PWM channels.
- Additional timer blocks.
- GPIO/PWM interrupt sources.
- Project-level DMA subsystem.
- Second interrupt controller / external PLIC.
- 10-bit I2C addressing, high-speed I2C modes, or other unsupported modes.
- SPI device-specific controller logic or undocumented advanced modes.
- Advanced PWM features such as complementary outputs, dead time, phase shifting or multi-channel operation.
- New PLLs or independent clock domains.
- Additional boot-memory regions.
- Automatic ICCM/DCCM/I-cache instantiation.

12.4 Change-Control Rule

A proposed feature is within scope only when it is explicitly represented in the block inventory, address map, signal list, register map, control/data path, reset/interrupt rules, and verification requirements. Otherwise Kiro shall stop at the nearest defined interface and not invent a new subsystem.

APPENDIX A - QUICK REFERENCE TABLES

A.1 One-Page Address Map

Range	Target	Select
0x0000_0000-0x0000_FFFF	IMEM	imem_sel
0x0001_0000-0x0001_FFFF	DMEM	dmem_sel
0x0002_0000-0x0002_00FF	UART	psel[0]
0x0002_0100-0x0002_01FF	Timer	psel[1]
0x0002_0200-0x0002_02FF	GPIO	psel[2]
0x0002_0300-0x0002_03FF	SPI	psel[3]
0x0002_0400-0x0002_04FF	I2C	psel[4]
0x0002_0500-0x0002_05FF	PWM	psel[5]
0x0002_0600-0xFFFF_FFFF	Reserved	none

A.2 One-Page Peripheral Signal Reference

Peripheral	Registers	External interface	IRQ
UART	UART_TXDATA/RXDATA/ STATUS/BAUD	TX/RX	UART_RX_IRQ
Timer	TMR_COUNT/COMPARE/ CTRL	none	TMR_IRQ
GPIO	GPIO_OUT/IN/DIR	GPIO[7:0]	None
SPI	SPI_TXDATA/RXDATA/CTRL/ STATUS/CLKDIV	SCLK/MOSI/MISO/CS_n	SPI_DONE_IRQ
I2C	I2C_TXDATA/RXDATA/CTRL/ STATUS/ADDR	SDA/SCL	I2C_DONE_IRQ
PWM	PWM_PERIOD/DUTY/CTRL	PWM_OUT	None

A.3 Processor-Side Bus Reference

Path	Master	Main signals
Instruction	IFU	haddr,hburst,hmastlock,hprot,hsize,htrans,h write,hdata,hready,hresp
Data	LSU	lsu_haddr,lsu_hburst,lsu_hmastlock,lsu_hpr ot,lsu_hsize,lsu_htrans,lsu_hwdata,lsu_hwri te,lsu_hrddata,lsu_hready,lsu_hresp
Debug	Debug Bus Master	sb_haddr,sb_hburst,sb_hmastlock,sb_hprot ,sb_hsize,sb_htrans,sb_hwdata,sb_hwrite,s b_hrddata,sb_hready,sb_hresp

APPENDIX B - FINAL CHECKLIST BEFORE RTL GENERATION

47. Confirm the exact selected VeeR EL2 RTL configuration and port names.
48. Confirm AHB-Lite is the actual processor-side configuration used.
49. Freeze the exact top-level clock/reset relationship: clk, PCLK, rst_l, PRESETn, dbg_rst_l.
50. Freeze the exact source-ID mapping for the four project IRQs if PIC source IDs are exposed to software.
51. Freeze the exact PIC priority values in firmware/integration, because the architecture makes priority programmable but does not mandate project numeric values here.
52. Freeze any VeeR Debug/JTAG ports according to the selected wrapper rather than inventing ports.
53. Freeze the exact local register reset values not explicitly stated in the architecture before RTL coding.
54. Run unit tests for each peripheral, then interconnect/address-isolation tests, then integrated software tests.
55. Run unmapped-access tests and verify that the CPU never hangs because of a missing response.
56. Run reset-during-transaction tests for SPI and I2C.
57. Run simultaneous-interrupt tests through the PIC.

58. Run debug halt/resume and supported system-bus access tests according to the selected VeeR configuration.
59. Reject any new feature that is not represented in this specification as a scope change.

APPENDIX C - FINAL SCOPE STATEMENT

The complete SoC is a VeeR EL2-based, memory-mapped embedded platform whose implementation is intentionally bounded. VeeR EL2 provides the processor architecture; IMEM and DMEM provide the project memories; the System Interconnect and Bus Adapter provide transaction routing/conversion; UART, Timer, GPIO, SPI, I2C and PWM provide the six project peripheral functions; the four defined peripheral interrupts use the VeeR external interrupt/PIC mechanism; and VeeR Debug remains processor-support infrastructure. Any subsystem not explicitly identified in this specification is not part of the baseline implementation.

13. DETAILED NON-FATAL ERROR AND ABNORMAL-CONDITION SPECIFICATION

This section expands the existing error boundary into a block-by-block fault/status reference. The project distinction is preserved: a peripheral protocol condition is first represented at the peripheral status/interrupt level; a processor architectural exception or NMI is generated only when the VeeR core itself defines that condition. Conditions whose exact implementation is not fixed by the project architecture are explicitly marked as verification/implementation decisions rather than silently promoted to mandatory hardware.

13.1 Error Classification

Class	Examples	Where visible	CPU exception/NMI?
Normal status condition	UART TX_BUSY, UART RX_VALID, Timer compare, SPI BUSY/DONE, I2C ACK/NACK/BUSY/DONE, GPIO input state, PWM output state	Peripheral STATUS/readback	No
Bus/interconnect abnormal access	Unmapped address, reserved address, invalid local register offset, unsupported access semantics	System interconnect / bus response	Only if the selected VeeR/core configuration converts it to an architectural fault
Debug-interface error	sdbusyerror, sberror, abstractcs.cmderr	Debug Module registers	No automatic CPU NMI
VeeR precise exception	Instruction access fault, illegal instruction, load/store misaligned, load/store access fault, M-mode ecall	mcause/mtval/mepc	Yes, architectural exception
VeeR local interrupt	Correctable error threshold, internal timer local interrupt	mip/mie/mcause	Interrupt, not project peripheral IRQ
VeeR NMI	D-bus store error, non-blocking load error, selected fast-interrupt/core faults, NMI pin assertion	mcause + VeeR NMI/error registers	Yes, fatal/non-maskable according to VeeR

13.2 System-Level Non-Fatal Conditions

The system interconnect and Bus Adapter must never leave a CPU-side request without a defined completion or error outcome. The supplied project architecture explicitly defines an unmapped-access response as zero read data with PREADY asserted after one wait cycle. This is a bus-safety requirement: an invalid address must not deadlock the processor.

```
CPU transaction
-> LSU
-> System Interconnect
-> Address Decoder
-> no target
-> defined unmapped response
-> PRDATA = 0x0000_0000 on peripheral-style side
-> PREADY asserted after one wait cycle
-> LSU response
-> CPU
```

Condition	Detection point	Required handling	Out of scope
Unmapped system address	Address Decoder	No target select; return defined unmapped response	Creating a new peripheral
Reserved region access	Address Decoder	No target select; use defined unmapped/reserved response	Repurposing reserved space
Undefined peripheral register offset	Local peripheral decoder	Return the defined reserved/invalid-register response	Inventing a new register
Transaction not ready	Peripheral PREADY / Adapter	Hold ACCESS as required; eventually complete or return	Infinite wait with no documented termination

		an error according to the selected bus implementation	
Response from wrong target	Response MUX	Prevent; only selected target may drive active read response	Broadcasting read data

13.3 Access-Size and Alignment Conditions

The VeeR core documents standard precise exceptions including load address misaligned, store/AMO address misaligned, load access fault and store/AMO access fault. The integration shall preserve those processor rules. The interconnect must not silently split a side-effecting peripheral access into multiple operations unless the selected VeeR/bus implementation explicitly defines such behaviour.

VeeR condition	mcause	Meaning for this SoC
Instruction access fault	0x0000_0001	Instruction fetch cannot complete normally
Illegal instruction	0x0000_0002	Fetch instruction is not legal for the implemented ISA/configuration
Breakpoint	0x0000_0003	Debug/software breakpoint trap
Load address misaligned	0x0000_0004	Load address violates required alignment
Load access fault	0x0000_0005	Load access is not accepted by the selected target/interface
Store/AMO address misaligned	0x0000_0006	Store/AMO address violates required alignment
Store/AMO access fault	0x0000_0007	Store/AMO cannot complete through the selected target/interface
Environment call from M-mode	0x0000_000B	M-mode software trap

SOURCE / SCOPE NOTE: The exact processor RTL decides how a bus error is converted into these standard faults. The project specification must not add a custom mcause value for UART, Timer, SPI or I2C.

13.4 Peripheral Abnormal/Status Conditions

Peripheral	Condition	Status owner	Interrupt relationship	Required scope
UART	TX_BUSY	UART_STATUS bit0	Not itself an interrupt source	Normal UART status
UART	RX_VALID	UART_STATUS bit1	UART_RX_IRQ is generated for the receive event	Status + defined IRQ
Timer	Compare match	TMR_COUNT/ TMR_COMPARE/ TMR_CTRL	TMR_IRQ	Timer event
GPIO	Input level / direction / output state	GPIO_IN/GPIO_DIR/ GPIO_OUT	No GPIO IRQ in baseline	Normal GPIO state
SPI	BUSY	SPI_STATUS bit0	No separate BUSY IRQ	Normal transfer status
SPI	DONE	SPI_STATUS bit1	SPI_DONE_IRQ	Transfer completion
I2C	BUSY	I2C_STATUS bit0	No separate BUSY IRQ	Transaction-in-progress status
I2C	ACK	I2C_STATUS bit1	No separate ACK IRQ	Protocol response state
I2C	DONE	I2C_STATUS bit2	I2C_DONE_IRQ	Transfer completion
PWM	Waveform configuration/output state	PWM registers / PWM_OUT	No PWM IRQ in baseline	Normal PWM operation

13.5 SPI and I2C Reset-Abort Conditions

PRESETn asserted

- > SPI active transfer aborted
- > SPI state/shift state reset

PRESETn asserted

- > I2C active transaction aborted
- > I2C FSM/TX/RX state reset

These reset-abort behaviours are required by the project reset architecture. They are not treated as protocol-error interrupts; reset is the controlling condition.

13.6 Debug Non-Fatal Errors

Debug condition	Register	Meaning	Recovery/handling
System-bus access attempted while busy	sbc.sbbusyerror	Overlapping debug system-bus request	Debugger clears the error and waits for the original operation to complete
Unsupported system-bus access size	sbc.sbserror	No access is performed for unsupported sbaccess encoding; VeeR documents code 4 for unsupported access size	Debugger selects a supported access size
Abstract command error	abstractcs.cmderr	Abstract command could not be executed as requested; busy is one documented case	Debugger reads/clears cmderr and retries when valid
Unsupported/unimplemented command or field	Command/implementation-specific status	Selected VeeR build does not implement requested option	Do not add unsupported functionality in project RTL

SOURCE / SCOPE NOTE: These errors do not automatically become nmi_int, reset, or a new project interrupt. They remain debug-interface conditions.

13.7 VeeR Fatal/NMI Boundary

The VeeR EL2 PRM explicitly documents fatal imprecise D-bus errors as NMIs. A D-bus store error has mcause 0xF000_0000 and a non-blocking D-bus load error has mcause 0xF000_0001; the first faulting address is captured in mdseac. The project must not manufacture these conditions inside a peripheral merely because a peripheral transaction failed.

NMI cause	mcause	Scope
NMI pin assertion	0x0000_0000	External NMI input; core-level only
D-bus store error	0xF000_0000	VeeR core D-bus fatal error
D-bus non-blocking load error	0xF000_0001	VeeR core D-bus fatal error
Fast Interrupt double-bit ECC error	0xF000_1000	VeeR fast-interrupt/core-specific feature; include only when present in selected VeeR build
Fast Interrupt DCCM region access error	0xF000_1001	VeeR core-specific feature; include only when present
Fast Interrupt non-DCCM region error	0xF000_1002	VeeR core-specific feature; include only when present

13.8 Correctable-Error Local Interrupt

VeeR counts I-cache parity/ECC errors and ICCM/DCCM correctable ECC errors in separate counters with programmable thresholds. When a counter reaches threshold, a correctable-error local interrupt is signaled. The PRM documents mceip and mceie at bit 30 of mip and mie, respectively, with mcause 0x8000_001E. This is a VeeR core feature and is not one of the four project peripheral IRQ sources.

```
I-cache / ICCM / DCCM correctable error
-> correctable-error counter
-> threshold reached
-> mceip/mceie path (bit 30)
-> mcause = 0x8000_001E
-> VeeR ISR
```

14. COMPLETE INTERRUPT SPECIFICATION - ALL RELEVANT LEVELS

This section separates every interrupt class relevant to the selected VeeR-based SoC into: (A) project peripheral interrupts that are part of the baseline SoC, (B) standard RISC-V/VeeR core interrupt inputs that exist independently of

the project peripheral IRQ list, and (C) VeeR local/NMI mechanisms that are core features and shall not be confused with project peripherals.

14.1 Baseline Project Peripheral IRQs

IRQ	Source event	Signal type	PIC path	Status source
TMR_IRQ	Timer compare event	Level-sensitive project source	Timer -> PIC gateway -> pending/priority/thresho ld -> external IRQ	Timer STATUS/CTRL
UART_RX_IRQ	UART receive event	Level-sensitive project source	UART -> PIC gateway -> pending/priority/thresho ld -> external IRQ	UART_STATUS/ RXDATA
SPI_DONE_IRQ	SPI transfer complete	Level-sensitive project source	SPI -> PIC gateway -> pending/priority/thresho ld -> external IRQ	SPI_STATUS/RXDATA
I2C_DONE_IRQ	I2C transfer complete	Level-sensitive project source	I2C -> PIC gateway -> pending/priority/thresho ld -> external IRQ	I2C_STATUS/RXDATA

The baseline architecture deliberately does not assign project IRQ outputs to GPIO or PWM. IMEM, DMEM, System Interconnect and Bus Adapter also do not create project peripheral IRQ outputs.

14.2 Standard VeeR/RISC-V Interrupt Inputs

Signal / class	Source	Typical machine-level representation	Project status
soft_int	VeeR software interrupt input	mip.msip / mie.msie; mcause 0x8000_0003	Core feature, not a project peripheral IRQ
timer_int	VeeR standard machine timer input	mip.mtip / mie.mtie; mcause 0x8000_0007	Core feature; distinct from TMR_IRQ
extintsrc_req[...]	VeeR external interrupt source vector	External interrupt; mcause 0x8000_000B	Used by project peripheral IRQ integration
nmi_int	VeeR non-maskable interrupt input	NMI path; NMI mcause set by exact NMI source	Core-only, not a project peripheral IRQ

14.3 VeeR Local Interrupts Documented by the PRM

Interrupt	mcause	Meaning	Project treatment
Machine internal timer 1 local interrupt	0x8000_001C	VeeR internal timer 1 local interrupt	Do not create as project peripheral
Machine internal timer 0 local interrupt	0x8000_001D	VeeR internal timer 0 local interrupt	Do not create as project peripheral
Machine correctable error local interrupt	0x8000_001E	Correctable I-cache/ICCM/DCCM threshold event	Core-local feature; include only if selected VeeR build exposes it

14.4 Project External Interrupt Decision Chain

```

Project IRQ source
-> extintsrc_req[source]
-> PIC gateway / synchronizer
-> pending state
-> per-source enable
-> priority vs threshold
-> compare all eligible sources
-> winning source ID / priority
-> PIC target
-> VeeR machine external interrupt
-> ISR
-> read source peripheral STATUS/data

```

```
-> clear/service source
-> mret
```

VeeR documents up to 255 external interrupt sources, 15 priority levels, per-source enable/priority, thresholding, interrupt claim, vectored external interrupt support, chaining and nesting. Only the four project peripheral sources are connected in the project baseline.

14.5 Machine External Interrupt CSR Path

```
Peripheral IRQ
-> PIC/external interrupt mechanism
-> mip.meip (bit 11)
-> mie.meie (bit 11) must permit delivery
-> machine external interrupt accepted
-> mcause = 0x8000_000B
-> mtvec / external vectoring mechanism
-> ISR
```

SOURCE / SCOPE NOTE: The exact availability/behaviour of mie/mip fields and external vectoring must match the selected VeeR RTL. The project architecture must not create separate mcause values for Timer, UART, SPI and I2C.

14.6 Interrupt Priority and Tie-Breaking

When multiple external sources are pending and eligible, VeeR uses source priority and threshold rules. If two eligible sources have the same priority, the documented VeeR selection rule uses the lower hardwired source ID as the tie-breaker. Numeric project priorities and the exact source-ID assignments must be frozen during integration; they are not invented in this document.

14.7 Interrupt Claim and Source Identification

```
Eligible pending sources
-> winner selected
-> meicpt captures winning claim ID/priority
-> meihap / vector mechanism resolves handler address
-> source-specific ISR
```

The ISR must still read the source peripheral STATUS register because the PIC identifies the interrupt source, while the peripheral status register describes the actual peripheral event.

14.8 Interrupt Nesting

VeeR supports interrupt nesting/chaining. During a nested-interrupt-capable handler, software can save the prior threshold, raise the current threshold to the handler priority, re-enable interrupts as required, service a higher-priority source, restore prior state, and execute mret. This is a VeeR/PIC capability; the project peripherals themselves do not implement nesting logic.

14.9 Interrupts Explicitly Out of Scope

- GPIO_IRQ - not part of the baseline project interrupt architecture.
- PWM_IRQ - not part of the baseline project interrupt architecture.
- IMEM_IRQ - not part of the baseline project interrupt architecture.
- DMEM_IRQ - not part of the baseline project interrupt architecture.
- Interconnect_IRQ - not part of the baseline project interrupt architecture.
- Bus_Adapter_IRQ - not part of the baseline project interrupt architecture.
- Debug-protocol-as-project-IRQ - not part of the baseline project interrupt architecture.
- Second global PLIC output - not part of the baseline project interrupt architecture.

15. COMPLETE REGISTER DESCRIPTION

This section is the detailed software-visible register reference. Project peripheral offsets are relative to each peripheral base address. VeeR machine CSRs and PIC/debug registers are a separate architectural namespace and must not be converted into project APB peripheral registers.

15.1 Project Peripheral Base Addresses

Peripheral	Base	End	Select
UART	0x0002_0000	0x0002_00FF	psel[0]
Timer	0x0002_0100	0x0002_01FF	psel[1]
GPIO	0x0002_0200	0x0002_02FF	psel[2]
SPI	0x0002_0300	0x0002_03FF	psel[3]
I2C	0x0002_0400	0x0002_04FF	psel[4]
PWM	0x0002_0500	0x0002_05FF	psel[5]

15.2 UART Registers

Offset	Register	Access	Detailed role	Important fields
0x00	UART_TXDATA	R/W	CPU writes transmit data into the UART TX path.	Transmit byte/data
0x04	UART_RXDATA	R	CPU reads received data from the UART RX path.	Received byte/data
0x08	UART_STATUS	R	Reports UART activity and receive availability.	bit0 TX_BUSY; bit1 RX_VALID
0x0C	UART_BAUD	R/W	Controls UART baud-rate configuration.	Baud configuration value

Control path: CPU STORE/LOAD -> LSU -> UART address window -> psel[0] -> Bus Adapter -> PADDR/PWRITE/PWDATA/PSEL/PENABLE -> selected UART register. Data return for reads is UART register -> PRDATA -> Adapter -> lsu_hrddata -> LSU.

15.3 Timer Registers

Offset	Register	Access	Detailed role	Important fields
0x00	TMR_COUNT	R/W	Current programmable/free-running timer count state.	Count value
0x04	TMR_COMPARE	R/W	Comparison threshold used to detect the timer event.	Compare threshold
0x08	TMR_CTRL	R/W	Enables the timer and its interrupt path.	bit0 ENABLE; bit1 IRQ_EN

Data path: TMR_COUNT/TMR_COMPARE values move through PWDATA into the timer state; TMR_COUNT can be read through PRDATA. Control path: TMR_CTRL -> timer operation -> compare logic -> TMR_IRQ.

15.4 GPIO Registers

Offset	Register	Access	Detailed role	Important fields
0x00	GPIO_OUT	R/W	Software-controlled output data.	GPIO output value
0x04	GPIO_IN	R	Current input pin state returned to software.	GPIO[7:0] input state
0x08	GPIO_DIR	R/W	Per-pin direction control.	bit=1 output; bit=0 input

The eight GPIO pins are GPIO[7:0]. GPIO_DIR controls whether each pin is used as an output or input. GPIO_OUT drives output logic, while GPIO_IN reads the input state. No GPIO interrupt register is included in the baseline map.

15.5 SPI Registers

Offset	Register	Access	Detailed role	Important fields
0x00	SPI_TXDATA	R/W	Software transmit data register loaded into the TX shift path.	Transmit data
0x04	SPI_RXDATA	R	Software readback of received data.	Received data
0x08	SPI_CTRL	R/W	Enables/configures the SPI serial controller.	bit0 EN; bit1 CPOL; bit2 CPHA
0x0C	SPI_STATUS	R	Reports transfer state.	bit0 BUSY; bit1 DONE
0x10	SPI_CLKDIV	R/W	Serial clock divider/configuration value.	SCLK divider

The SPI registers control a full-duplex master. A software write to SPI_TXDATA supplies the transmit payload; SPI_CTRL/configuration causes the FSM to perform the serial transfer; SPI_CLKDIV controls SCLK timing; SPI_RXDATA provides the received value; SPI_STATUS reports BUSY/DONE; SPI_DONE_IRQ reports completion to the VeeR external interrupt path.

15.6 I2C Registers

Offset	Register	Access	Detailed role	Important fields
0x00	I2C_TXDATA	R/W	Transmit byte/data supplied to the I2C serial engine.	Transmit byte/data
0x04	I2C_RXDATA	R	Received byte returned to software.	Received byte/data
0x08	I2C_CTRL	R/W	Controls START/STOP and read/write direction.	bit0 START; bit1 STOP; bit2 RW
0x0C	I2C_STATUS	R	Reports transaction state and ACK/DONE information.	bit0 BUSY; bit1 ACK; bit2 DONE
0x10	I2C_ADDR	R/W	Holds the external target address.	7-bit target address

I2C_ADDR is the external I2C slave address and is not the same as PADDR. The CPU/system address selects the I2C controller; I2C_ADDR selects the external device during the serial address phase.

15.7 PWM Registers

Offset	Register	Access	Detailed role	Important fields
0x00	PWM_PERIOD	R/W	Sets the PWM period/top count value.	Period value
0x04	PWM_DUTY	R/W	Sets the duty-cycle compare threshold.	Duty compare value
0x08	PWM_CTRL	R/W	Controls PWM enable and output polarity.	bit0 ENABLE; bit1 POLARITY

The PWM generator uses a free-running counter, PWM_PERIOD and PWM_DUTY. Comparator logic determines PWM_OUT from the counter-to-duty relationship, repeating over the programmed period. The project baseline does not add a PWM interrupt register.

15.8 VeeR Standard Machine-Level CSRs - Relevant Set

CSR	Address	Description	Project relevance
misa	0x301	ISA and extensions; VeeR notes writes ignored	Identify implemented ISA/configuration

mstatus	0x300	Machine status	Global machine status and interrupt/trap state
mie	0x304	Machine interrupt-enable	Enables machine interrupt classes; external enable is bit 11 in the documented VeeR path
mtvec	0x305	Machine trap-handler base address and mode	Trap/interrupt entry
mscratch	0x340	Scratch register for machine trap handlers	ISR temporary storage
mepc	0x341	Machine exception program counter	Saved return PC
mcause	0x342	Machine trap cause	Identifies exception/interrupt class
mtval	0x343	Machine bad address or instruction	Additional fault information when applicable
mip	0x344	Machine interrupt pending	Pending machine interrupt classes; external pending is bit 11 in the documented path
mhartid	0xF14	Hardware thread ID	Hart identity

SOURCE / SCOPE NOTE: The exact software-visible CSR implementation must match the selected VeeR EL2 RTL. These are source-derived VeeR CSRs, not project APB registers.

15.9 VeeR mcause Values Relevant to Error/Interrupt Debugging

mcause value	Class	Description
0x0000_0001	Exception	Instruction access fault
0x0000_0002	Exception	Illegal instruction
0x0000_0003	Exception	Breakpoint
0x0000_0004	Exception	Load address misaligned
0x0000_0005	Exception	Load access fault
0x0000_0006	Exception	Store/AMO address misaligned
0x0000_0007	Exception	Store/AMO access fault
0x0000_000B	Exception	Environment call from M-mode
0x8000_0003	Interrupt	Machine software interrupt
0x8000_0007	Interrupt	Machine timer interrupt
0x8000_000B	Interrupt	Machine external interrupt
0x8000_001C	Interrupt	Machine internal timer 1 local interrupt
0x8000_001D	Interrupt	Machine internal timer 0 local interrupt
0x8000_001E	Interrupt	Machine correctable error local interrupt
0x0000_0000	NMI	NMI pin assertion
0xF000_0000	NMI	D-bus store error
0xF000_0001	NMI	D-bus non-blocking load error
0xF000_1000	NMI	Fast Interrupt double-bit ECC error
0xF000_1001	NMI	Fast Interrupt DCCM region access error
0xF000_1002	NMI	Fast Interrupt non-DCCM region error

15.10 VeeR PIC Registers

Register	Address / mapping	Detailed role
mpiccfg	PIC_base_addr + 0x3000	PIC configuration including priority-order behaviour
meiplS	PIC_base_addr + S*4, S=1..255	External interrupt source priority level
meipX	PIC_base_addr + 0x1000 + X*4	Pending information for source groups
meieS	PIC_base_addr + 0x2000 + S*4	External interrupt source enable
meipt	CSR 0xBC9	External interrupt priority threshold
meivt	CSR 0xBC8	External interrupt vector table base
meihap	CSR 0xFC8	External interrupt handler address pointer / dispatch support
meicpct	CSR 0xBCA	Captures winning interrupt claim ID and priority
meicidpl	CSR 0xBCB	Winning/current claim source priority information

meicurpl	CSR 0xBCC	Current interrupt priority level
meigwctrlS	PIC_base_addr + 0x4000 + S*4	Gateway configuration for source S when configurable gateway exists
meigwclrS	PIC_base_addr + 0x5000 + S*4	Gateway clear trigger; WAR0/write-only semantics documented by VeeR

The VeeR PRM also provides PIC memory-mapped register address ranges. A register is only present for a source when the corresponding configurable gateway is instantiated. The project shall use only the gateway/source configuration present in the selected VeeR build.

15.11 Debug Module DMI Register Set

DMI offset	Register	Detailed role
0x04	data0	Abstract-command data register 0
0x05	data1	Abstract-command data register 1
0x10	dmcontrol	Hart/debug control: haltreq, resumereq, ndmreset, dmactive and supported fields
0x11	dmstatus	Debug Module and hart status
0x16	abstractcs	Abstract-command status/error
0x17	command	Abstract command selection
0x18	abstractauto	Abstract-command auto-execution control
0x38	sbcs	Debug system-bus control/status
0x39	sbaddress0	Debug system-bus address
0x3C	sbddata0	Debug system-bus data lower portion
0x3D	sbddata1	Debug system-bus data upper portion
0x40	haltsum0	Hart halt summary

15.12 VeeR Debug CSRs

CSR	Address	Role
tselect	0x7A0	Debug/trace trigger selection
tdata1	0x7A1	Trigger data/control; VeeR documents mcontrol in this space
tdata2	0x7A2	Second trigger data
dcsr	0x7B0	Debug control and status
dpc	0x7B1	Debug program counter

16. APPLICATIONS - COMPLETE PROJECT-SCOPE USE CASES

Applications below are demonstrations of the existing project blocks. They do not introduce new RTL blocks. Any motor driver, LED driver, sensor, EEPROM, display or other load shown in an application is external hardware connected to the project output/interface.

16.1 Embedded UART Console

Blocks: VeeR + UART + Debug (optional).

Allowed path: CPU/LSU -> UART_TXDATA/UART_STATUS; UART_RX -> UART_RXDATA/UART_RX_IRQ -> PIC -> CPU.

Application: Firmware console, command input, diagnostic messages and simple monitoring.

16.2 Periodic Task Scheduler

Blocks: VeeR + Timer.

Allowed path: CPU programs TMR_COUNT/TMR_COMPARE/TMR_CTRL -> timer event -> TMR_IRQ -> PIC -> CPU ISR -> software task.

Application: Periodic sampling, control-loop scheduling, timeouts implemented in software.

16.3 Digital Switch and LED Control

Blocks: VeeR + GPIO.

Allowed path: CPU writes GPIO_DIR/GPIO_OUT; CPU reads GPIO_IN.

Application: Simple digital input/output, switches, LEDs, enable lines.

16.4 SPI Sensor / Flash / Display Link

Blocks: VeeR + SPI.

Allowed path: CPU -> SPI_TXDATA/CTRL/CLKDIV -> SPI FSM -> SCLK/MOSI/CS_n; MISO -> SPI_RXDATA; SPI_DONE_IRQ signals completion.

Application: External SPI sensors, serial flash devices and compatible displays, subject to software protocol handling.

16.5 I2C Sensor Interface

Blocks: VeeR + I2C.

Allowed path: CPU -> I2C_ADDR/I2C_TXDATA/I2C_CTRL -> START/address/RW/ACK/data/STOP; SDA -> I2C_RXDATA; I2C_DONE_IRQ -> CPU.

Application: Low-pin-count digital sensors and similar standard-mode I2C devices.

16.6 EEPROM Interface

Blocks: VeeR + I2C.

Allowed path: CPU programs the I2C target address and data transaction; I2C performs standard-mode bus sequencing; software interprets EEPROM-specific command/addressing.

Application: Low-speed nonvolatile memory access.

16.7 Motor Speed Control

Blocks: VeeR + PWM + external driver.

Allowed path: CPU -> PWM_PERIOD/PWM_DUTY/PWM_CTRL -> counter/comparator -> PWM_OUT -> external motor driver.

Application: Programmable motor-drive command generation.

16.8 LED Dimming

Blocks: VeeR + PWM + external LED/driver.

Allowed path: CPU changes PWM_DUTY while PWM hardware continuously generates PWM_OUT.

Application: Variable brightness control.

16.9 Sensor-Driven Actuator

Blocks: I2C or SPI + VeeR + PWM.

Allowed path: Sensor data -> CPU -> software scaling/control -> PWM_DUTY -> PWM_OUT -> external actuator driver.

Application: Closed-loop or open-loop embedded control demonstration.

16.10 Integrated Educational SoC Platform

Blocks: VeeR + IMEM + DMEM + interconnect + UART/Timer/GPIO/SPI/I2C/PWM.

Allowed path: Software exercises all memory-mapped blocks through the single defined architecture; interrupts demonstrate VeeR external interrupt servicing.

Application: Teaching, RTL integration practice and end-to-end verification.

16.11 Debug-Assisted Validation

Blocks: VeeR + Debug Module + selected SoC memory/core features.

Allowed path: Debugger -> JTAG/DTM/DMI -> Debug Module -> halt/resume or supported debug memory access.

Application: Bring-up, register inspection, controlled execution and fault diagnosis.

16.12 Application Scope Rules

- Applications may combine existing peripherals through CPU software.
- Applications shall not create direct Timer-to-PWM, Timer-to-GPIO, UART-to-PWM or I2C-to-PWM hardware links unless an explicit architecture change adds such a path.
- External loads are not new SoC RTL blocks.
- Application firmware may configure registers, poll status, service the four project IRQs and use the Debug interface for development.
- Device-specific command protocols belong in software unless an explicit new hardware controller is approved.

17. ERROR / INTERRUPT / REGISTER / APPLICATION VERIFICATION MATRIX

Block / feature	Positive checks	Abnormal checks	Expected outcome
UART	TX/RX register read/write, TX activity, RX event	BUSY/RX_VALID transitions, invalid offset	Defined STATUS; UART_RX_IRQ only for receive event
Timer	COUNT, COMPARE, CTRL, periodic compare	Disabled timer, reset, invalid offset	TMR_IRQ only when configured condition occurs
GPIO	OUT/IN/DIR and all 8 pins	Direction misconfiguration, reserved offset	Correct digital state; no GPIO IRQ
SPI	TX/RX, CTRL, CLKDIV, serial pins, DONE	BUSY handling, reset during transfer, reserved offset	Transfer completes; SPI_DONE_IRQ; no bus lock
I2C	START/STOP, address, RW, TX/RX, ACK/NACK, DONE	NACK, reset during transaction, reserved offset	Defined status; I2C_DONE_IRQ for completion; no bus lock
PWM	PERIOD/DUTY/CTRL, waveform	Disabled state, reset, reserved offset	Defined waveform; no PWM IRQ
Interconnect	Address decode, PSEL, response mux, PREADY	Unmapped/reserved access, wrong-target response	Defined response; no lock-up
PIC	Enable/priority/threshold/claim/source ID	Simultaneous IRQs, disabled source, low priority below threshold	Correct winning source and CPU external interrupt
Debug	DMI registers, halt/resume, supported memory access	sbbusyerror, sberror, cmderr	Error visible in correct debug register; no arbitrary CPU NMI
Clock/reset	Startup, reset release, reset propagation	Reset during SPI/I2C, debug reset separation	Known states; specified abort/disable behaviour

18. FINAL SCOPE LOCK - ERRORS, INTERRUPTS, REGISTERS AND APPLICATIONS

The expanded sections above are additive detail for the existing specification. They do not change the original block inventory or memory map. Kiro shall use the project registers and signal names already defined, and shall treat VeeR-specific CSRs/PIC/Debug registers as processor architecture rather than project peripheral registers. Where the project source does not fix a low-level implementation detail, the implementation must stop at the nearest defined interface and record the unresolved item for RTL sign-off instead of inventing functionality.

PROJECT PERIPHERAL IRQs ONLY:

TMR_IRQ
UART_RX_IRQ
SPI_DONE_IRQ
I2C_DONE_IRQ

NO BASELINE PROJECT IRQs FROM:

GPIO

PWM
IMEM
DMEM
Interconnect
Bus Adapter

PROJECT REGISTER MAP:

UART -> 0x0002_0000-0x0002_00FF
Timer -> 0x0002_0100-0x0002_01FF
GPIO -> 0x0002_0200-0x0002_02FF
SPI -> 0x0002_0300-0x0002_03FF
I2C -> 0x0002_0400-0x0002_04FF
PWM -> 0x0002_0500-0x0002_05FF

VeeR MACHINE EXTERNAL IRQ:

Project IRQ -> VeeR external interrupt/PIC
-> mcause class 0x8000_000B

ERROR PRINCIPLE:

status condition != CPU exception by default
debug error != NMI
only VeeR-defined core faults use VeeR exception/NMI mechanisms

19. SOURCE BASIS FOR THIS EXPANSION

Primary project source: the supplied RISC-V SoC Specification Document / architecture material. It defines the project block inventory, memory map, register map, peripheral bus, four project peripheral IRQs, applications and verification direction. Source basis in the working document: Section 1 introduction and Sections 3-12 plus its appendices.

VeeR EL2 source: RISC-V VeeR EL2 Programmer's Reference Manual Rev. 1.4 (2022-12-22). Used for the VeeR machine CSR address map, PIC capabilities/registers, Debug Module register architecture, reset distinctions, core-level interrupt/NMI causes and documented debug error conditions.

SOURCE / SCOPE NOTE: Important: the exact faculty-provided VeeR RTL/configuration remains authoritative for which optional VeeR interfaces, CSRs, PIC gateways and Debug ports are actually instantiated. This document intentionally marks such items as configuration-dependent when the source does not guarantee their presence.